Lab 5: A UART TRX Controller

Yuval Zecharia

04/11/2026

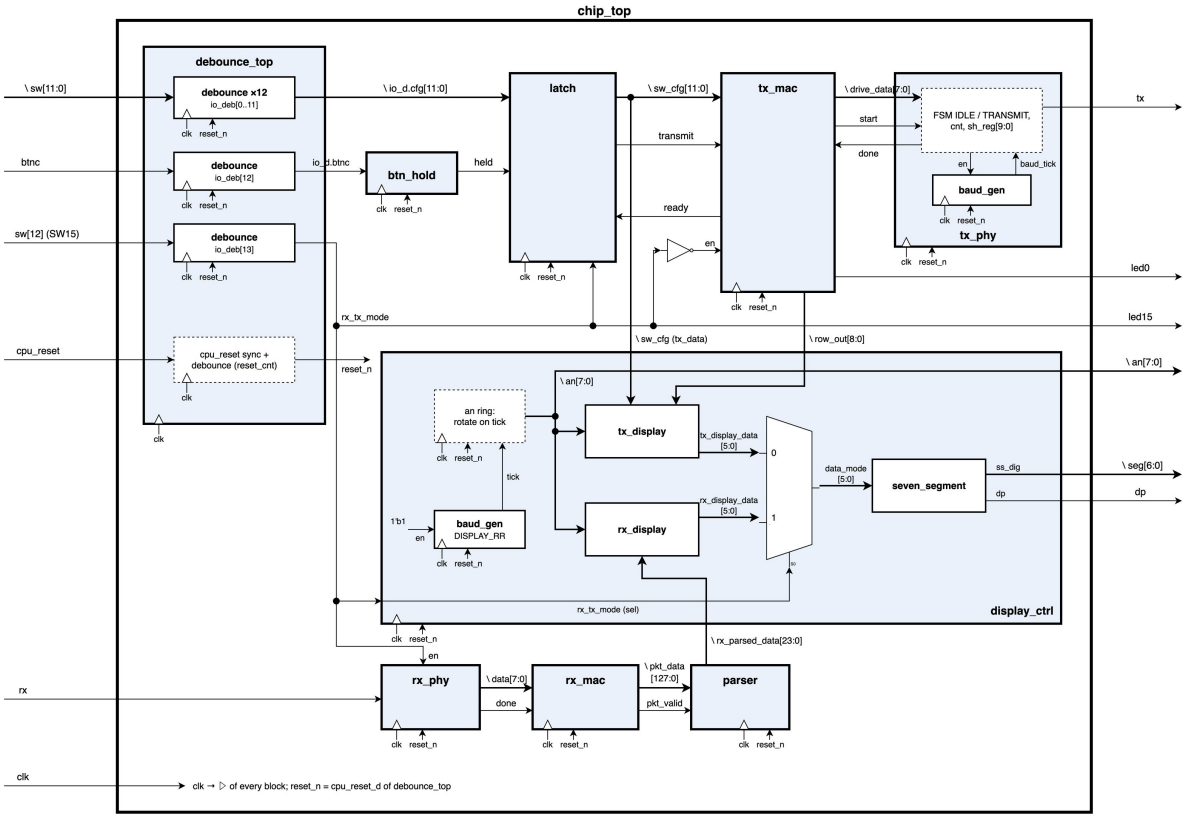
1. מטרת המעבדה

מטרת המעבדה היא לממש ב-SystemVerilog, על לוח Nexys A7-100T, בקר UART דו-כיווני (TRX) שמשדר ל-PC ריבוע של בית שנבחר במתגים ומקבל מה-PC הודעות בפורמט {Rddd,Cddd,Vddd}. המתג SW15 בוחר בין מצב TX, שבו תצוגת ה-Seven Segment מראה את תצורת השידור, למצב RX, שבו היא מראה את השורה, העמודה וערך הפיקסל שהתקבלו.

2. תהליך התכנון

2.1. ארכיטקטורה

המערכת בנויה משני נתיבים, TX ו-RX, כניסות ה-io של הלוח עוברות שלב debouncing ומשם משורשרות לשאר ה-chip. מתג 15 של הלוח קובע את mode הפעולה של ה-chip. אם הוא 1 מצב rx אחרת מצב tx. ממשק ה-io של chip_top מפורט בטבלה 1.



איור 1: מבנה chip_top: תת-המודולים והאותות ביניהם.

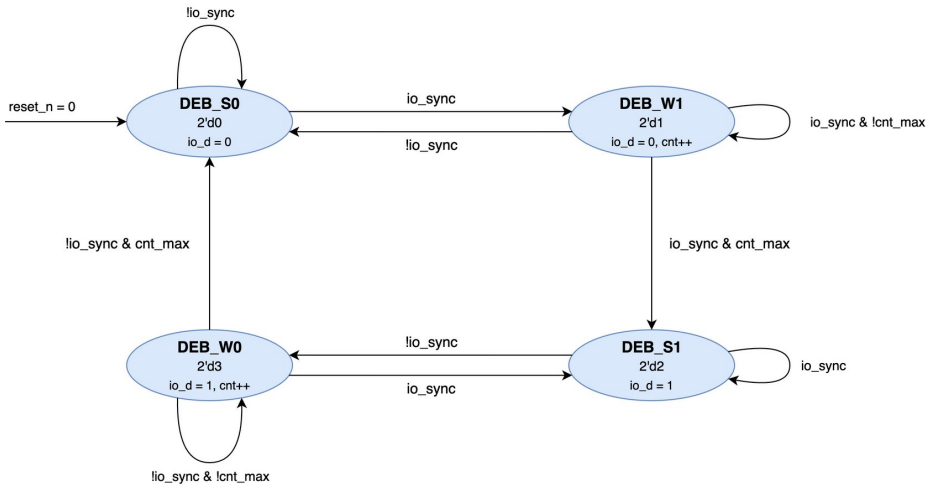
2.2. ממשק chip_top

טבלה 1: ממשק chip_top.

Port name	Direction	Type	Description
clk	Input	logic	100 MHz system clock (E3)
cpu_reset	Input	logic	CPU_RESETN push-button, active low (C12)
sw	Input	logic [12:0]	{SW15, SW14, SW13, SW9 … SW0} = {mode, size[1:0], delay[1:0], data[7:0]}
btnc	Input	logic	Center push-button: held 1 s latches the switches and starts TX (N17)
rx	Input	logic	UART line from the PC (UART_TXD_IN, C4)
tx	Output	logic	UART line to the PC (UART_RXD_OUT, D4); 1 when idle
an	Output	logic [7:0]	Digit anodes, active low, one digit at a time
seg	Output	logic [6:0]	Cathodes {CA … CG} of the active digit, active low
dp	Output	logic	Decimal point, active low: on at AN3 in TX mode (T1 = 0.5 …)
led0	Output	logic	LED[0]: toggles on every data byte (byte 0 on, byte 1 off, …)
led15	Output	logic	LED[15] = rx_tx_mode (1 = RX, 0 = TX)

2.3. מכונות המצבים

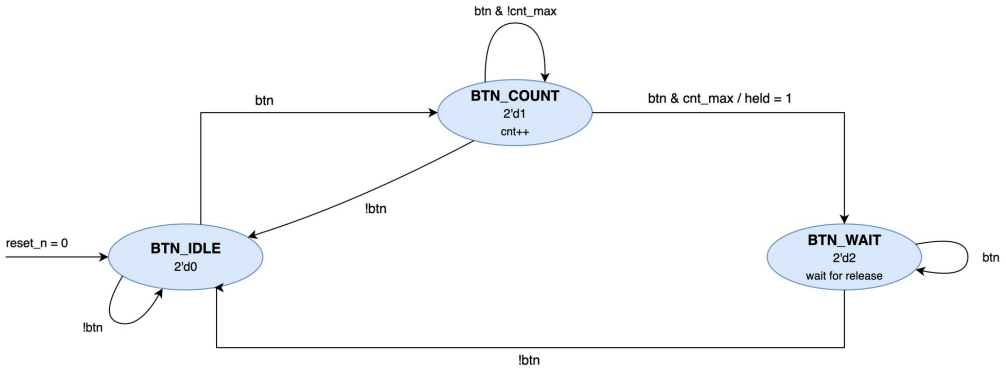
בתכנון שש מכונות מצבים (איורים 7–2 וטבלאות 7–2). הקודים בדיאגרמות הם ערכי ה-enum שב-lab5_pkg, והסינתזה מקודדת אותם מחדש one-hot (FSM_ENCODING=auto): ב-btn_hold, ב-tx_mac וב-rx_mac, קידוד בינארי של שני ביטים ב-debounce וב-rx_phy, ו-tx_phy נשאר דלגלג יחיד. כשאף תנאי יציאה אינו מתקיים, המכונה נשארת במצבה (הלולאות העצמיות בדיאגרמות).



איור 2: דיאגרמת המצבים של debounce (מופע אחד לכל כניסה).

טבלה 2: טבלת המעברים של debounce.

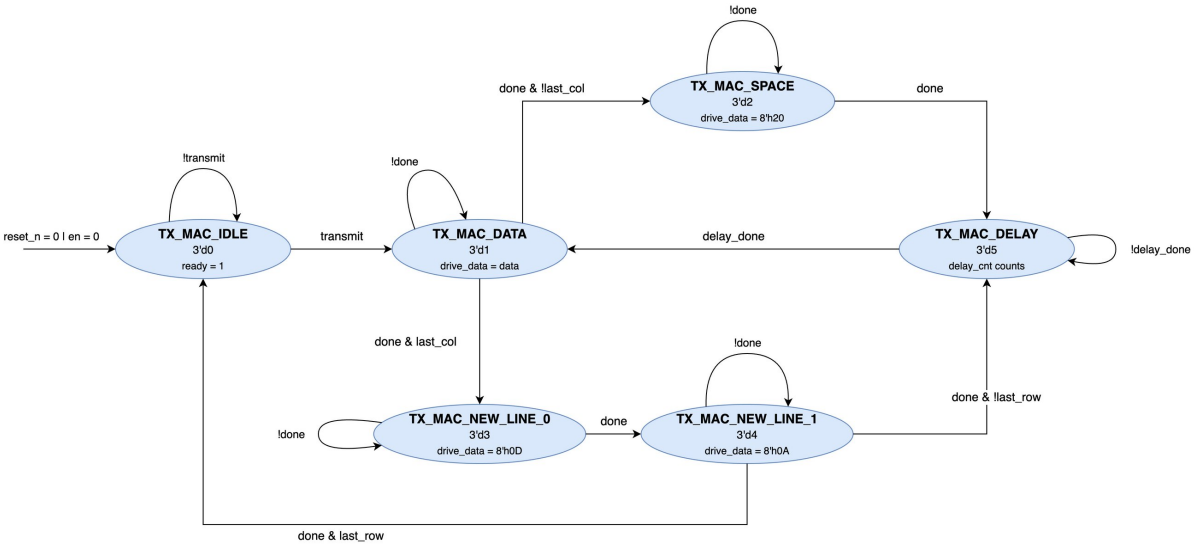
Current state	Condition	Next state	Registered on the transition
DEB_S0 (0)	io_sync	DEB_W1	cnt = 0
DEB_W1 (1)	!io_sync	DEB_S0	cnt = 0 (a glitch)
DEB_W1 (1)	io_sync & cnt_max	DEB_S1	cnt = 0; io_d = 1 from now on
DEB_S1 (2)	!io_sync	DEB_W0	cnt = 0
DEB_W0 (3)	io_sync	DEB_S1	cnt = 0 (a glitch)
DEB_W0 (3)	!io_sync & cnt_max	DEB_S0	cnt = 0; io_d = 0 from now on



איור 3: דיאגרמת המצבים של btn_hold.

טבלה 3: טבלת המעברים של btn_hold.

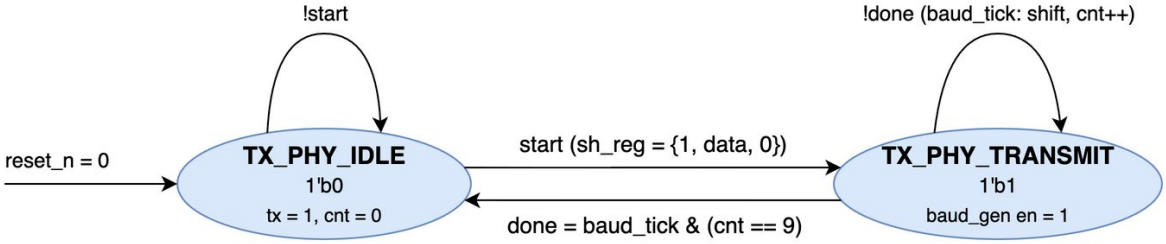
Current state	Condition	Next state	Registered on the transition
BTN_IDLE (0)	btn	BTN_COUNT	cnt = 0
BTN_COUNT (1)	btn & !cnt_max	BTN_COUNT	cnt++
BTN_COUNT (1)	!btn	BTN_IDLE	cnt = 0
BTN_COUNT (1)	btn & cnt_max	BTN_WAIT	cnt = 0; held = 1 in this clock
BTN_WAIT (2)	!btn	BTN_IDLE	–



איור 4: דיאגרמת המצבים של tx_mac.

טבלה 4: טבלת המעברים של tx_mac.

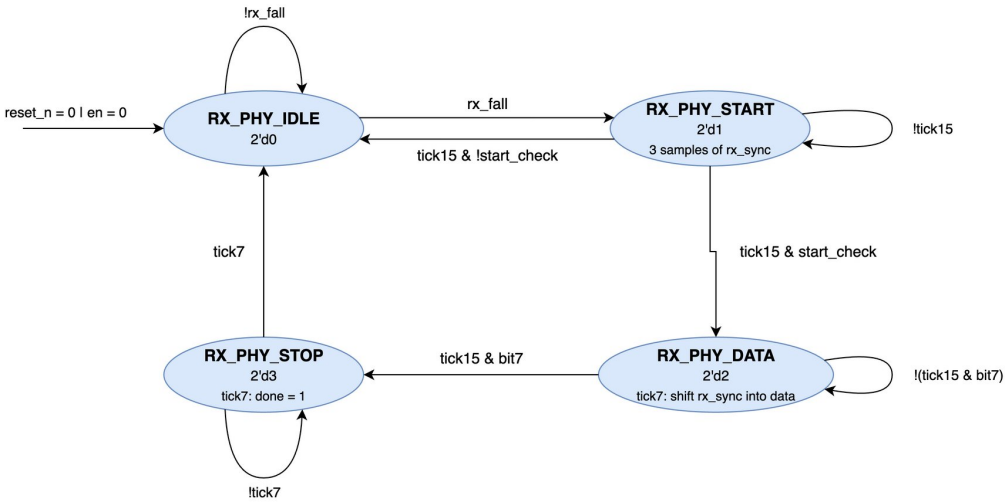
Current state	Condition	Next state	Registered on the transition
TX_MAC_IDLE (0)	transmit	TX_MAC_DATA	start, drive_data = data, led0 toggles; col = row = 0
TX_MAC_DATA (1)	done & !last_col	TX_MAC_SPACE	start, drive_data = 8'h20
TX_MAC_DATA (1)	done & last_col	TX_MAC_NEW_LINE_0	start, drive_data = 8'h0D
TX_MAC_SPACE (2)	done	TX_MAC_DELAY	col++, delay_cnt = 0
TX_MAC_NEW_LINE_0 (3)	done	TX_MAC_NEW_LINE_1	start, drive_data = 8'h0A
TX_MAC_NEW_LINE_1 (4)	done & !last_row	TX_MAC_DELAY	row++, col = 0, delay_cnt = 0
TX_MAC_NEW_LINE_1 (4)	done & last_row	TX_MAC_IDLE	row++ (row_out = N)
TX_MAC_DELAY (5)	delay_done	TX_MAC_DATA	start, drive_data = data, led0 toggles
any state	en = 0 (RX mode)	TX_MAC_IDLE	–



איור 5: דיאגרמת המצבים של tx_phy.

טבלה 5: טבלת המעברים של tx_phy.

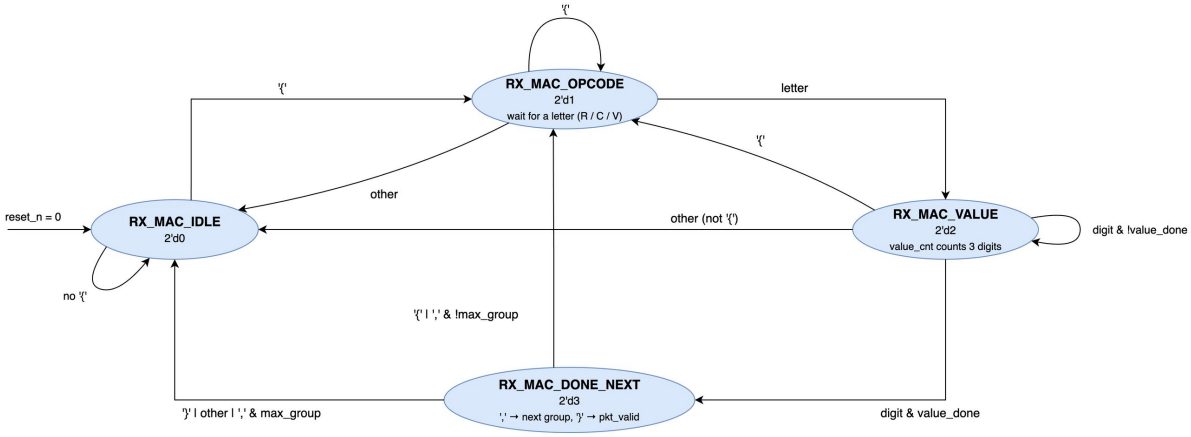
Current state	Condition	Next state	Registered on the transition
TX_PHY_IDLE (0)	start	TX_PHY_TRANSMIT	sh_reg = {1, data, 0}: tx = 0 (start bit)
TX_PHY_TRANSMIT (1)	baud_tick & cnt < 9	TX_PHY_TRANSMIT	sh_reg = {1, sh_reg[9:1]}, cnt++
TX_PHY_TRANSMIT (1)	done = baud_tick & cnt == 9	TX_PHY_IDLE	cnt = 0; tx = 1



איור 6: דיאגרמת המצבים של rx_phy.

טבלה 6: טבלת המעברים של rx_phy.

Current state	Condition	Next state	Registered on the transition
RX_PHY_IDLE (0)	rx_fall	RX_PHY_START	tick_cnt = tick16_cnt = 0: the ticks align to the falling edge
RX_PHY_START (1)	tick15 & start_check	RX_PHY_DATA	bit_cnt = 0
RX_PHY_START (1)	tick15 & !start_check	RX_PHY_IDLE	– (a glitch, not a start bit)
RX_PHY_DATA (2)	tick7 (stays)	RX_PHY_DATA	data = {rx_sync, data[7:1]}
RX_PHY_DATA (2)	tick15 & bit7	RX_PHY_STOP	–
RX_PHY_STOP (3)	tick7	RX_PHY_IDLE	done = 1 for one clock
any state	en = 0 (TX mode)	RX_PHY_IDLE	–



איור 7: דיאגרמת המצבים של rx_mac.

טבלה 7: טבלת המעברים של rx_mac (כל מעבר דורש done, כלומר בית שהתקבל).

Current state	Condition	Next state	Registered on the transition
RX_MAC_IDLE (0)	'{'	RX_MAC_OPCODE	pkt_data = {0…0, '{'}, group_cnt = 0
RX_MAC_OPCODE (1)	letter	RX_MAC_VALUE	shift the byte in, value_cnt = 0
RX_MAC_OPCODE (1)	'{' / other	RX_MAC_OPCODE / RX_MAC_IDLE	restart with '{' / drop the packet
RX_MAC_VALUE (2)	digit & !value_done	RX_MAC_VALUE	shift in, value_cnt++
RX_MAC_VALUE (2)	digit & value_done	RX_MAC_DONE_NEXT	shift in (the third digit)
RX_MAC_VALUE (2)	'{' / other	RX_MAC_OPCODE / RX_MAC_IDLE	restart with '{' / drop the packet
RX_MAC_DONE_NEXT (3)	',' & !max_group	RX_MAC_OPCODE	shift in, group_cnt++
RX_MAC_DONE_NEXT (3)	'}'	RX_MAC_IDLE	shift in; pkt_valid = 1 for one clock
RX_MAC_DONE_NEXT (3)	'{'	RX_MAC_OPCODE	restart with '{'
RX_MAC_DONE_NEXT (3)	other, ',' & max_group	RX_MAC_IDLE	– (a fourth group is rejected)

2.4. שיקולי תכנון

- קצב השידור (BAUD_RATE=57600, מהדרישות): baud_gen ב-tx_phy סופר COUNT מחזורים ומפיק baud_tick רק כשה-en שלו פעיל, במצב TX_PHY_TRANSMIT. COUNT מחושב בחלוקה שלמה, ולכן הקצב בפועל מהיר מעט מ-57600 (משוואה 1), בשגיאה יחסית של 0.0064% (משוואה 2). פריים של 10 ביטים נמשך 173.6µs (משוואה 3).

$$COUNT = \lfloor \frac{100\ MHz}{57600} \rfloor = \lfloor 1736.11 \rfloor = 1736,\ f_{actual} = \frac{100\ MHz}{1736} = 57603.69\ bps \quad (1)$$

$$error = \frac{f_{actual} - BAUD_RATE}{BAUD_RATE} = \frac{57603.69 - 57600}{57600} = +0.0064\% \quad (2)$$

$$T_{frame} = 10 \cdot COUNT \cdot T_{clk} = 10 \cdot 1736 \cdot 10\ ns = 173.6\ \mu s \quad (3)$$

- קליטה (אותו BAUD_RATE): rx_phy מחלק כל ביט ל-16 ticks (oversampling). TICK_CNT מעוגל כלפי מטה ל-108, ולכן ביט במקלט קצר ב-0.47% מהביט הנומינלי (משוואה 4). הירידה של ה-start bit מאפסת את המונים, וכל ביט נדגם ב-tick7, באמצע הביט. ההסטה המצטברת עד ה-stop bit, שנדגם 9.5 ביטים אחרי הירידה, היא 77 מחזורים בלבד, 4.4% מביט (משוואה 5). start bit מאומת רק אחר 3 דגימות רציפות שוות באמצע הביט הראשון.

$$TICK_CNT = \lfloor \frac{100\ MHz}{57600 \cdot 16} \rfloor = \lfloor 108.51 \rfloor = 108 \Rightarrow T_{bit,RX} = 16 \cdot 108 = 1728\ clk \quad (4)$$

$$\Delta_{stop} = 9.5 \cdot (1736.11 - 1728) \approx 77\ clk \approx 0.044 \cdot T_{bit} \quad (5)$$

- מבנה ההודעה: rx_mac בודק את המבנה תוך כדי קליטה ({, אות, שלוש ספרות, פסיק בין קבוצות, }) ומכניס כל בית מתאים לרגיסטר הזזה של 128 ביטים, כאורך הודעה מלאה של 16 בתים (משוואה 6). אילו אותיות התקבלו בודק ה-parser, ולא rx_mac.

$$N_{packet} = 2 + 3 \cdot (1 + 3) + (3 - 1) = 16\ bytes = 128\ bit \quad (6)$$

- פענוח: ה-parser בודק שהאותיות הן R, C ו-V (גם קטנות) ושהפסיקים במקומם, ממיר כל ערך לבינארי (משוואה 7) ובודק שאינו גדול מ-255, כי שורה, עמודה וערך פיקסל הם ברוחב 8 ביטים. כאשר מתקבלת הודעה שגויה ה-parser ממשיך להציג את ההודעה התקינה האחרונה.

$$value = 100 \cdot d_2 + 10 \cdot d_1 + d_0 \le 999 \;\Rightarrow\; \lceil log_2 1000 \rceil = 10\ bit \tag{7}$$

- תצוגה (DISPLAY_RR): רענון של 500Hz לכל אחת משמונה הספרות נותן DISPLAY_RR=4000, ו-baud_gen של התצוגה סופר 25000 מחזורים בין tick ל-tick (משוואה 8). במצב TX השדות הם T0 הבית, T1 ההשהיה, T2 הגודל ו-T3 מספר השורות. dp מחובר ל-an[3], ולכן T1 מציג את ההשהיה ביחידות של 0.0, 0.5, 1.0 או 2.0: 100ms. במצב RX השדות הם T0 ערך הפיקסל, T1 שני מקפים, T2 העמודה ו-T3 השורה, והערכים ב-hex.

$$DISPLAY_RR = 8 \cdot f_{refresh} = 8 \cdot 500\ Hz = 4000 \;\Rightarrow\; COUNT = \frac{f_{clk}}{DISPLAY_RR} = \frac{100\ MHz}{4000} = 25000 \tag{8}$$

- debounce וסנכרון (DEB_COUNT): כל אחת מ-14 הכניסות (12 ביטי התצורה, BTNC ו-SW15) עוברת מופע debounce משלה: סנכרון בשני דלגלגים ומכונת מצבים עם מונה יציבות של 20ms (משוואה 9). גם CPU_RESETN עובר סנכרון ומונה יציבות משלו, ומפיק את reset_n של כל התכנון.

$$DEB_COUNT = f_{clk} \cdot 20\ ms = 2 \cdot 10^6 \;\Rightarrow\; \lceil log_2(2 \cdot 10^6) \rceil = 21\ bit \tag{9}$$

- החזקה של שנייה (BTN_HOLD_COUNT): btn_hold סופר BTN_HOLD_COUNT מחזורים כל עוד הכפתור לחוץ (משוואה 10), ובסופם מפיק held אחד לכל לחיצה. המצב BTN_WAIT מחכה לשחרור, ולכן החזקה ארוכה אינה מפעילה שידור נוסף.

$$BTN_HOLD_COUNT = f_{clk} \cdot 1\ s = 10^8 \;\Rightarrow\; \lceil log_2 10^8 \rceil = 27\ bit \tag{10}$$

- השהיה בין בתים (DELAY_50MS_VAL): 50ms הם DELAY_50MS_VAL מחזורים, ו-100ms ו-200ms הם פי 2 ופי 4. רוחב delay_cnt נקבע לפי הגדול שבהם (משוואה 11). ב-delay=0 ההשהיה היא מחזור אחד במצב TX_MAC_DELAY. ההשהיה נכנסת אחרי כל רווח ואחרי סוף כל שורה, חוץ מהשורה האחרונה.

$$DELAY_50MS_VAL = f_{clk} \cdot 50\ ms = 5 \cdot 10^6 \;\Rightarrow\; \lceil log_2(4.5 \cdot 10^6) \rceil = 25\ bit \tag{11}$$

- מונה השידור: אחרי בית נתון שאינו אחרון בשורה נשלח 0x20, ואחרי האחרון נשלחים 0x0D ו-0x0A; מכאן מספר הבתים בריצה (משוואה 12). מספר בתי הנתון מגיע עד 65536 ואינו נכנס בשתי ספרות hex, ולכן T3 מציג את row_out, מספר השורות שנשלחו, ו-FF כש-row_out[8]=1, כלומר כש-row_out מגיע ל-256. מאותה סיבה T2 מציג FF עבור N=256. LED0 מתהפך רק בבתי נתון.

$$N_{bytes} = N \cdot (N + (N-1) + 2) = N \cdot (2N + 1) \;\Rightarrow\; N = 32\text{: } 2080,\ N = 256\text{: } 131328 \tag{12}$$

- בחירת המצב: latch דוגם את התצורה רק כש-rx_tx_mode!, ready & held, כך שלחיצה באמצע ריצה או במצב RX אינה משנה אותה, ו-transmit עולה באותו מחזור שבו התצורה נדגמת. tx_mac מקבל en=rx_tx_mode!, ולכן מעבר ל-RX באמצע ריצה מחזיר אותו מיד ל-TX_MAC_IDLE. rx_phy מקבל en=rx_tx_mode ונשאר ב-RX_PHY_IDLE במצב TX. כך בכל מצב פעיל רק נתיב אחד, כנדרש (אין full duplex).
- אילוצי תזמון: בתכנון שעון אחד (create_clock על clk). הכניסות מסונכרנות בתוך התכנון והיציאות איטיות ביחס לשעון, ולכן קובץ ה-XDC מגדיר להן set_false_path.
- סימולציה: כשההגדרה SIM קיימת (רק בסימולציה), lab5_definitions.svh נותן ערכים שמקצרים את הריצה: BAUD_RATE הוא 6.25MHz; DEB_COUNT הוא 10; BTN_HOLD_COUNT הוא 20; DELAY_50MS_VAL הוא 10; DISPLAY_RR הוא 10MHz, tick כל 10 מחזורים.

2.5. קוד המקור

קוד כל המודולים, החבילה lab5_pkg, קובץ ההגדרות ושורות ה-XDC הפעילות מובאים בנספח א'.

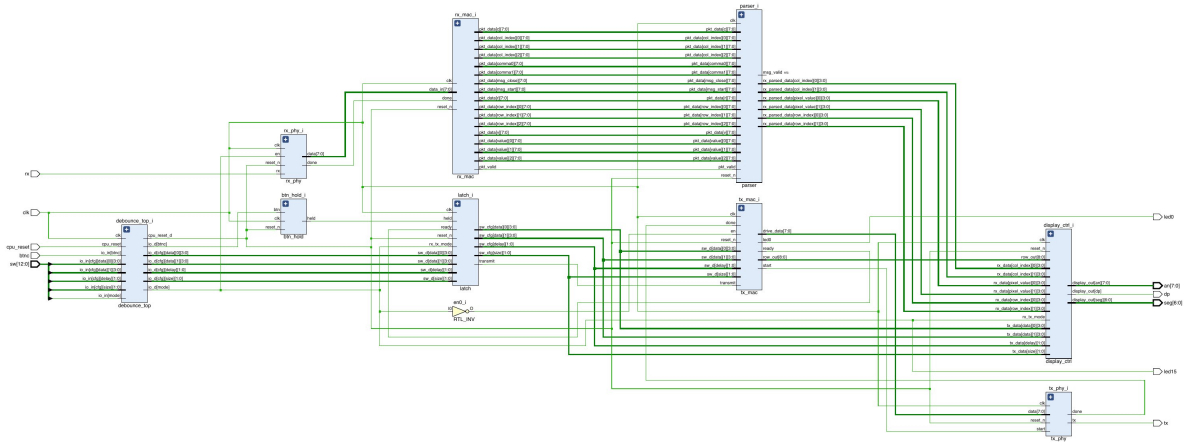
3. בדיקת Lint

טבלה 8: תוצאות בדיקות ה-Lint וה-Elaborated Design.

Check	Scope	Result
RTL Linter (Run Linter)	chip_top + all sub-modules	0 violations
Elaborated design – Report DRC	chip_top	0 checks
Elaborated design – Report Methodology	chip_top	0 checks

ה-Linter רץ על chip_top עם כל תת-המודולים, ודוחות ה-DRC וה-Methodology של ה-Elaborated Design נקיים (טבלה 8). סכמת ה-RTL מוצגת באיור 8.

3.1. סכמת RTL (Elaborated Design)



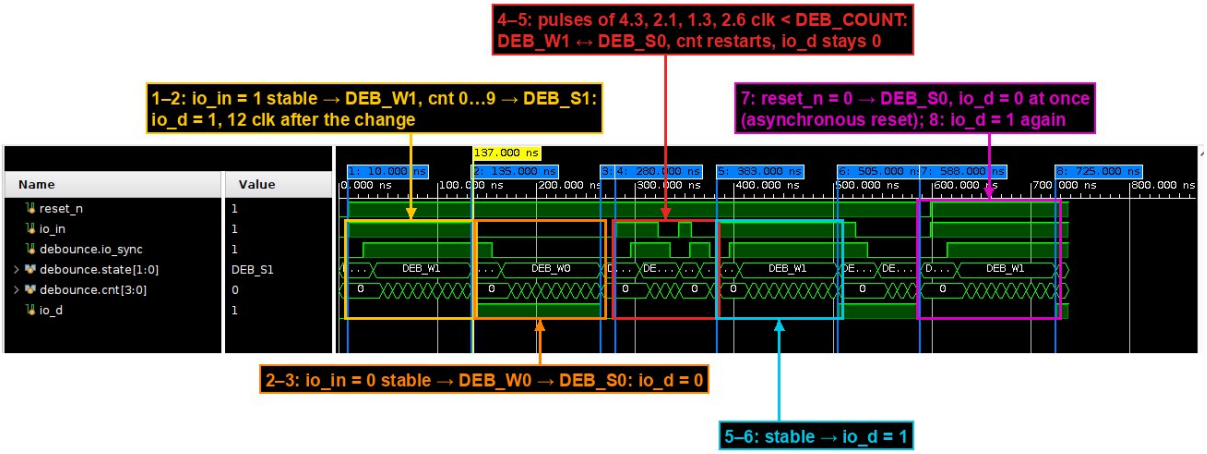
איור 8: סכמת ה-RTL של chip_top: תשעת תת-המודולים, המהפך RTL_INV על ה-en של tx_mac ופורטי ה-struct של תת-המודולים מפורקים לשדות.

4. תוצאות סימולציה

כל הטסטבנצ'ים רצו עם ההגדרה SIM, ו-assert שנכשל מודפס בלוג כ-ERROR. בכל סעיף מובאות תוצאת הריצה וצורות הגל, וההסבר מופיע על גבי האיור.

4.1. tb_debounce – בדיקת debounce

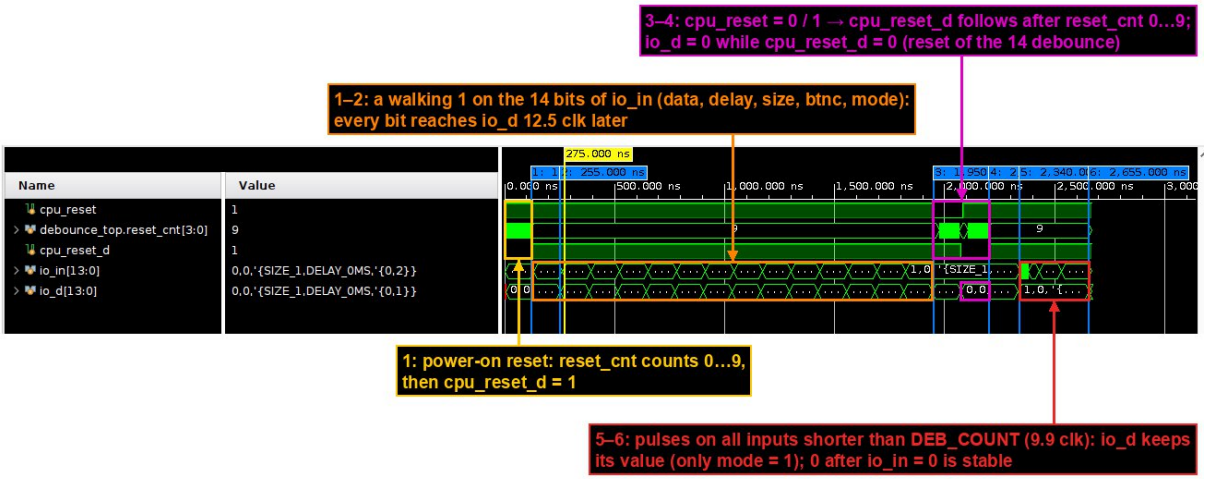
הריצה הסתיימה ב-738ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 9.



איור 9: tb_debounce – כל הריצה: שינוי יציב, פולסים קצרים ואיפוס.

4.2. tb_debounce_top – בדיקת debounce_top

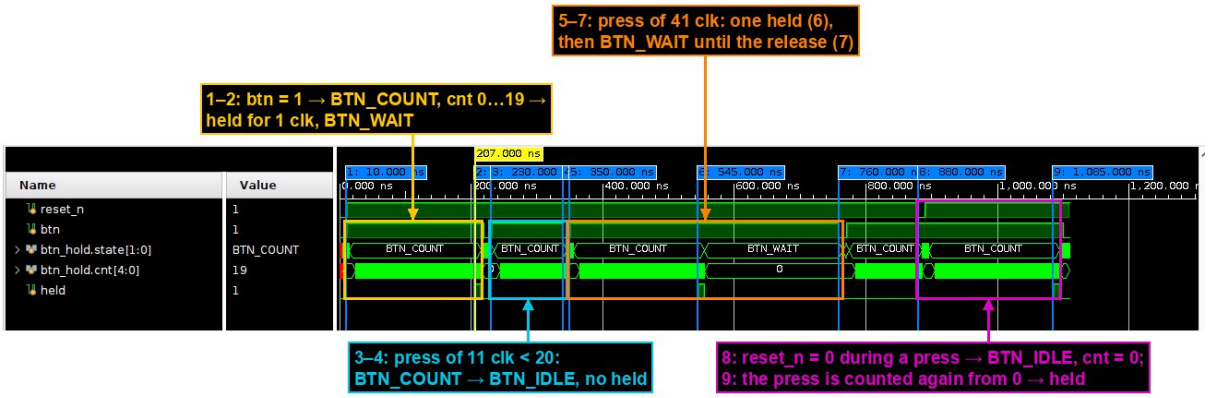
הריצה הסתיימה ב-2670ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 10.



איור 10: tb_debounce_top – כל הריצה: power-on reset, 1 רץ על 14 הכניסות, איפוס ופולסים קצרים.

4.3. tb_btn_hold – בדיקת btn_hold

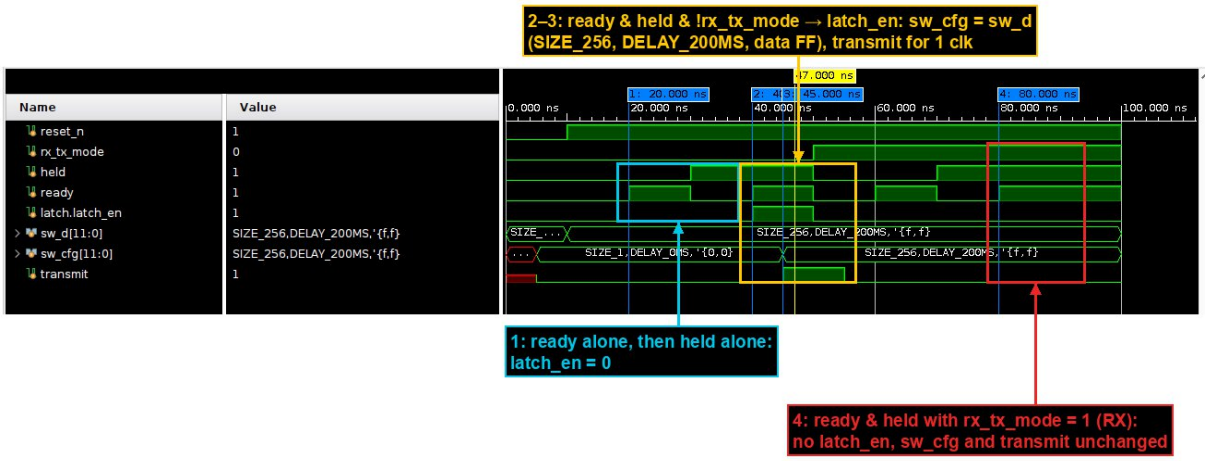
הריצה הסתיימה ב-1110ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 11.



איור 11: tb_btn_hold – כל הריצה: לחיצה רגילה, קצרה, ארוכה ואיפוס באמצע.

4.4. tb_latch – בדיקת latch

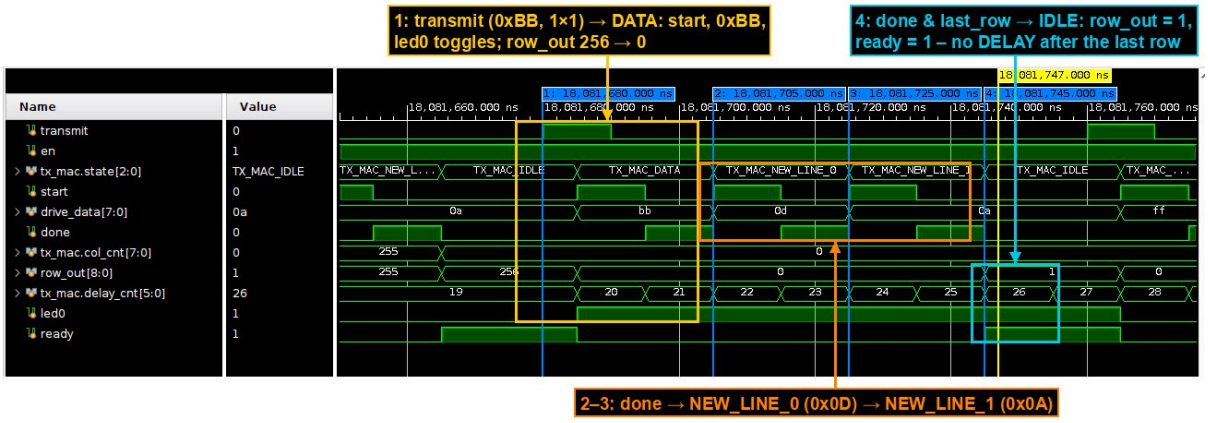
הריצה הסתיימה ב-100ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 12.



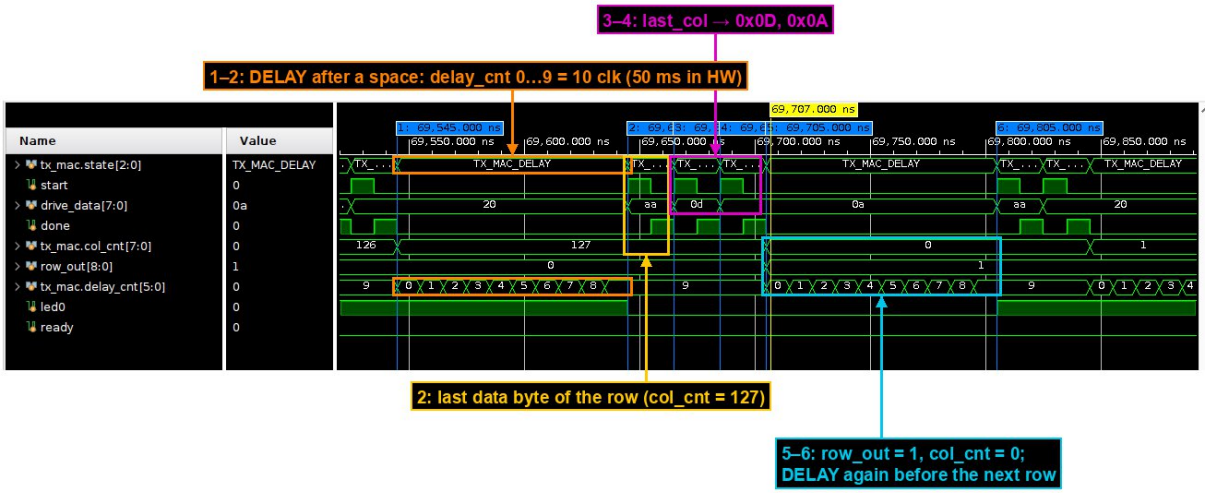
איור 12: tb_latch – כל הריצה: כל הצירופים של rx_tx_mode, held ו-ready.

4.5. tb_tx_mac – בדיקת tx_mac

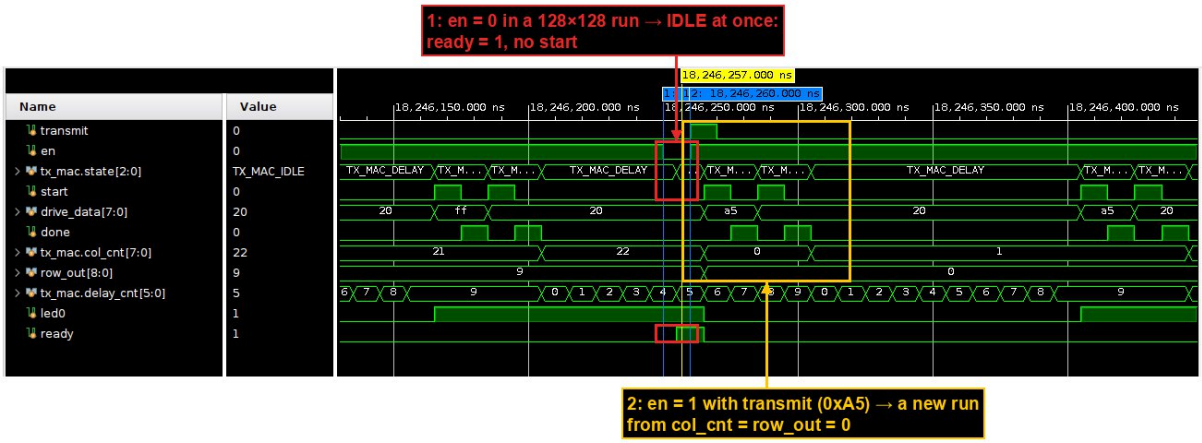
הריצה הסתיימה ב-20.54ms בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 13–15.



איור 13: tb_tx_mac – זום: ריצת 1×1 שלמה של 0xBB.



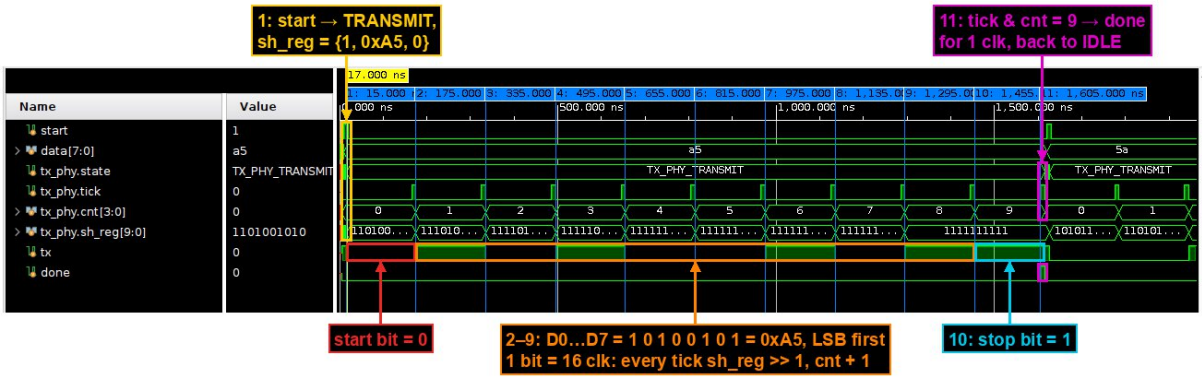
איור 14: tb_tx_mac – זום: סוף השורה הראשונה בריצת 128×128 עם delay=1.



איור 15: tb_tx_mac – זום: en=0 באמצע ריצה ותחילת ריצה חדשה.

4.6. tb_tx_phy – בדיקת tx_phy

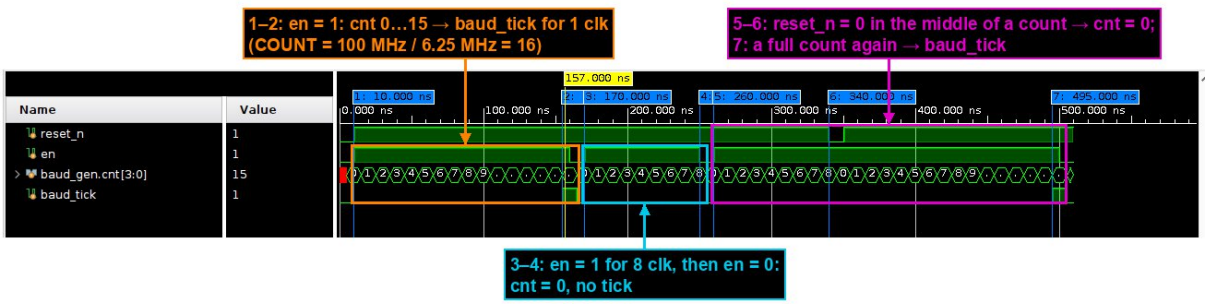
הריצה הסתיימה ב-7450ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 16.



איור 16: tb_tx_phy – זום: שידור הבית 0xA5.

4.7. tb_baud_gen – בדיקת baud_gen

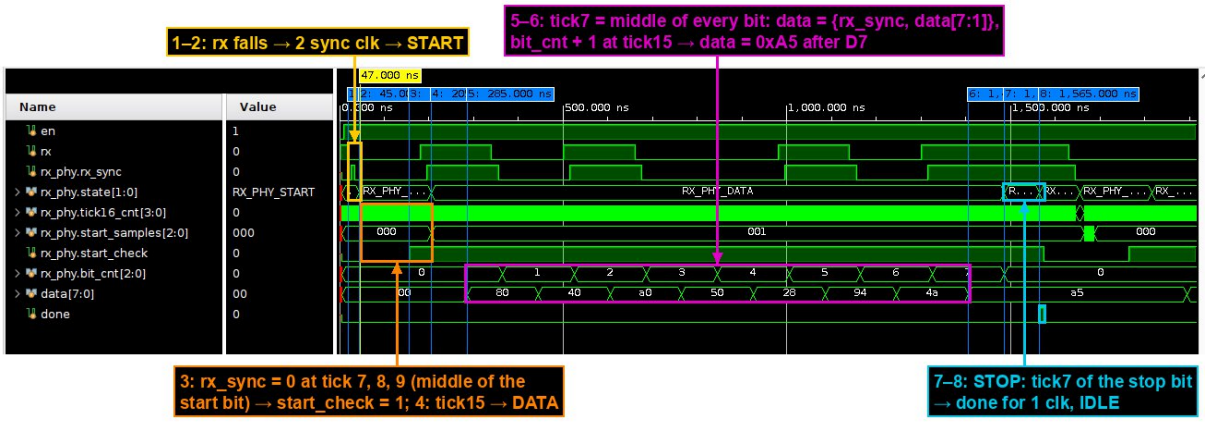
הריצה הסתיימה ב-510ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 17.



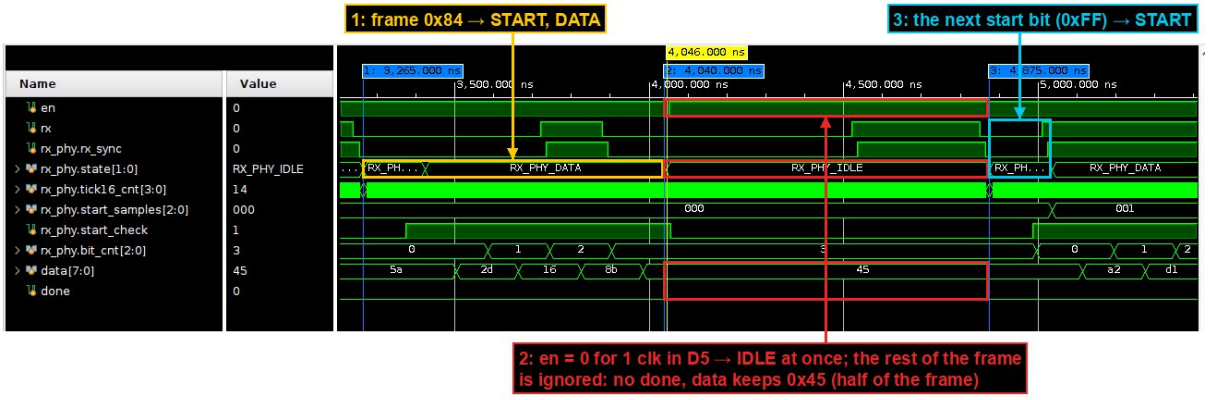
איור 17: tb_baud_gen – כל הריצה: ספירה מלאה, en=0 באמצע ואיפוס.

4.8. tb_rx_phy – בדיקת rx_phy

הריצה הסתיימה ב-8070ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 18–19.



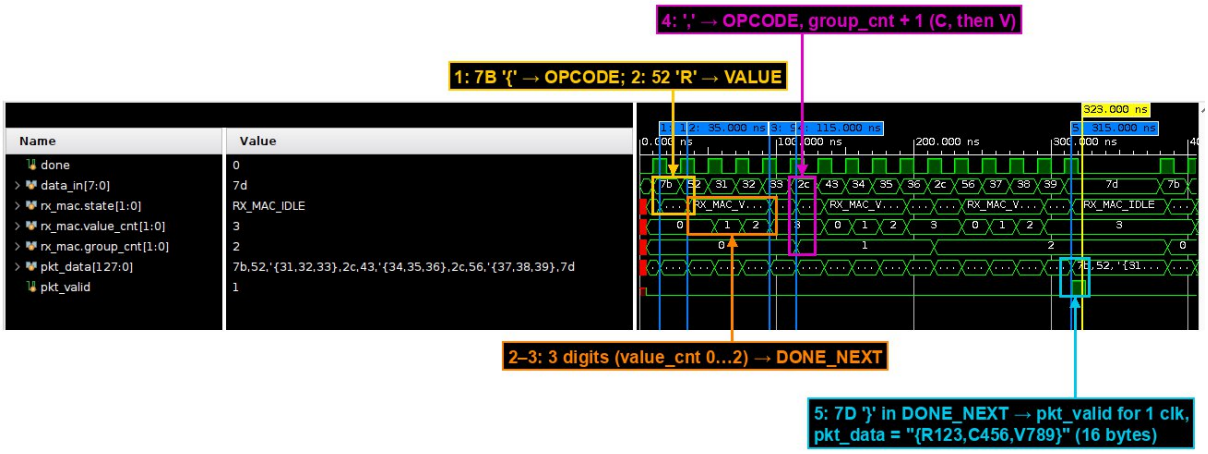
איור 18: tb_rx_phy – זום: קליטת הבית 0xA5.



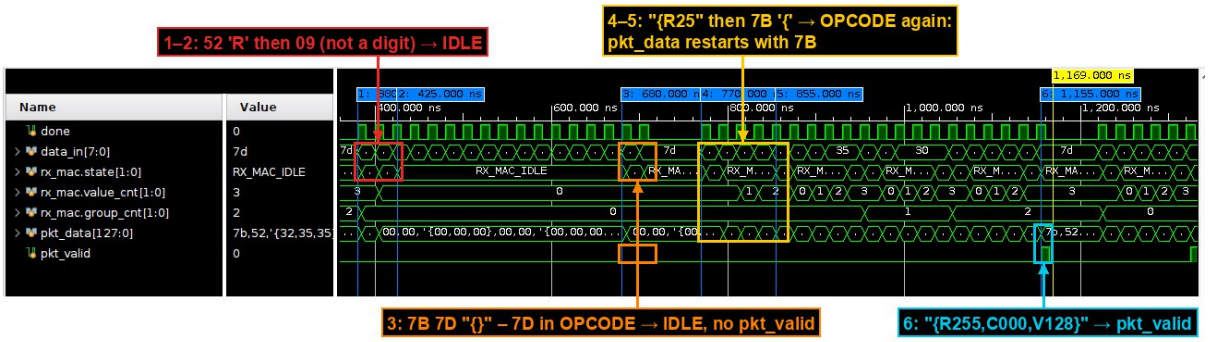
איור 19: tb_rx_phy – זום: en=0 באמצע הפריים 0x84.

4.9. tb_rx_mac – בדיקת rx_mac

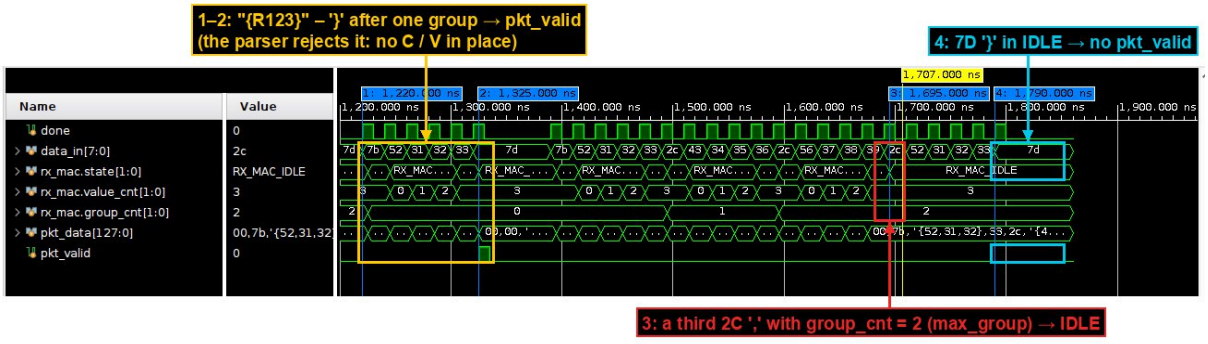
הריצה הסתיימה ב-1860ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיורים 20–22.



איור 20: tb_rx_mac – זום: ההודעה {R123,C456,V789}.



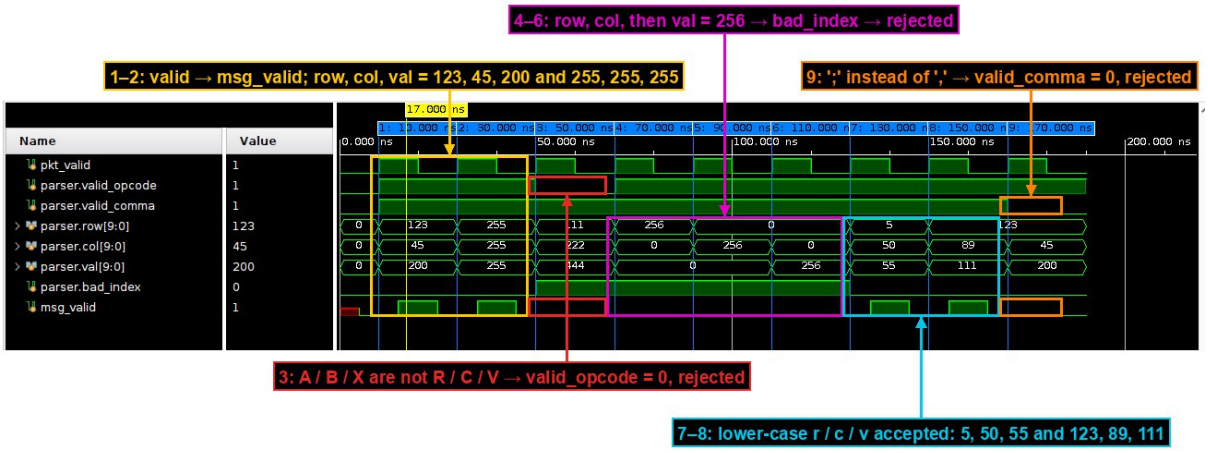
איור 21: tb_rx_mac – זום: תו שאינו ספרה, הודעה ריקה ו-{ באמצע הודעה.



איור 22: tb_rx_mac – זום: הודעה של קבוצה אחת והודעה של ארבע קבוצות.

4.10. tb_parser – בדיקת parser

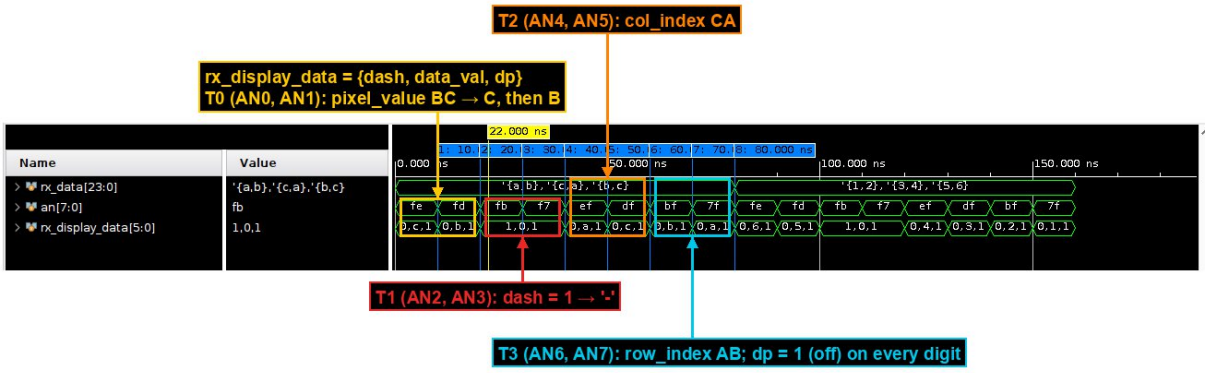
הריצה הסתיימה ב-190ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 23.



איור 23: tb_parser – כל הריצה: תשע הודעות.

4.11. tb_rx_display – בדיקת rx_display

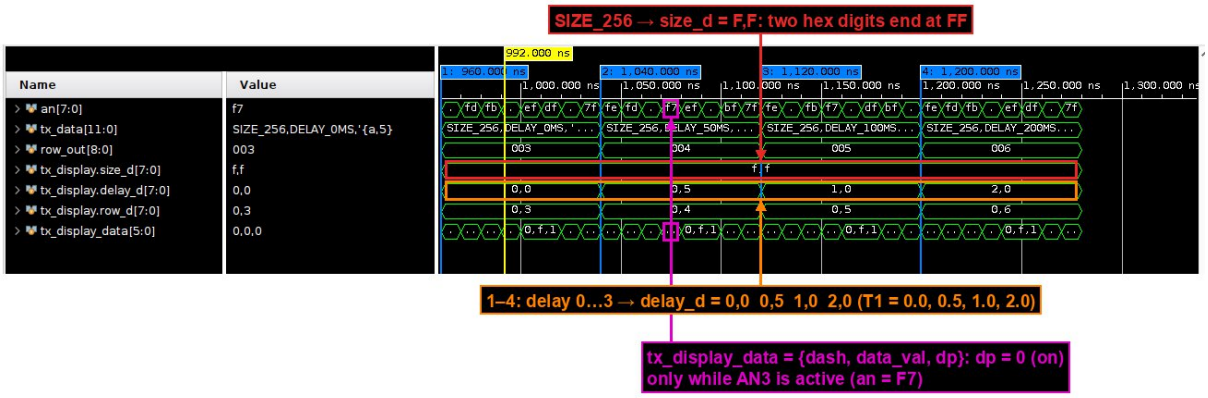
הריצה הסתיימה ב-160ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 24.



איור 24: tb_rx_display – כל הריצה: שני ערכים, שמונה אנודות.

4.12. tb_tx_display – בדיקת tx_display

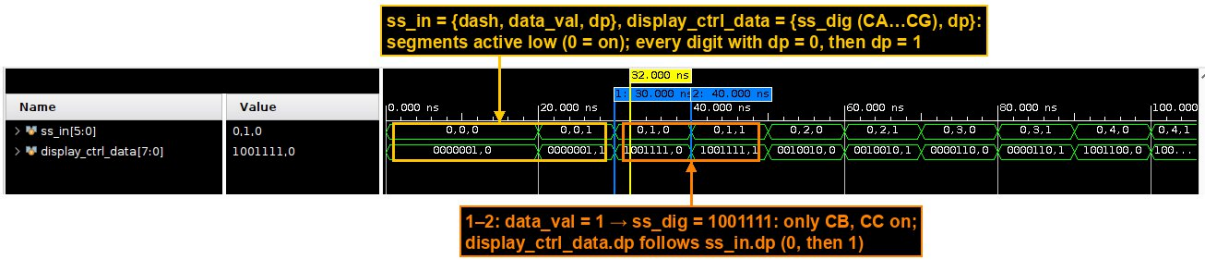
בטסטבנץ' זה אין assertions, ולכן התוצאה נבדקת בצורות הגל באיור 25, והיא כצפוי. הריצה הסתיימה ב-1280ns.



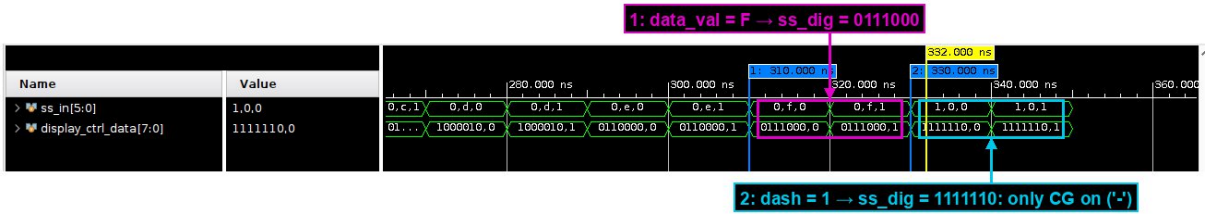
איור 25: tb_tx_display – זום: size=3 (SIZE_256) עם ארבע ההשהיות.

4.13. tb_seven_segment – בדיקת seven_segment

בטסטבנץ' זה אין assertions, ולכן התוצאה נבדקת בצורות הגל באיורים 26–27, והיא כצפוי. הריצה הסתיימה ב-350ns.



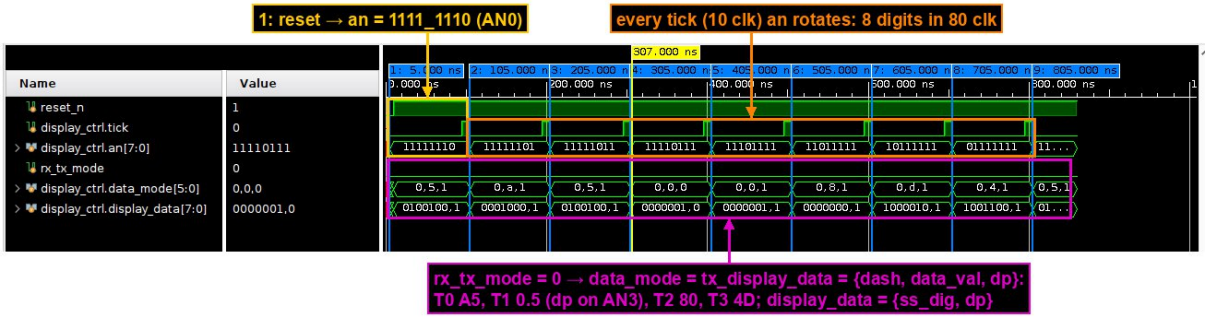
איור 26: tb_seven_segment – זום: הספרות 0 עד 3, כל אחת עם dp=0 ועם dp=1.



איור 27: tb_seven_segment – זום: הספרות D עד F והמקף.

4.14. tb_display_ctrl – בדיקת display_ctrl

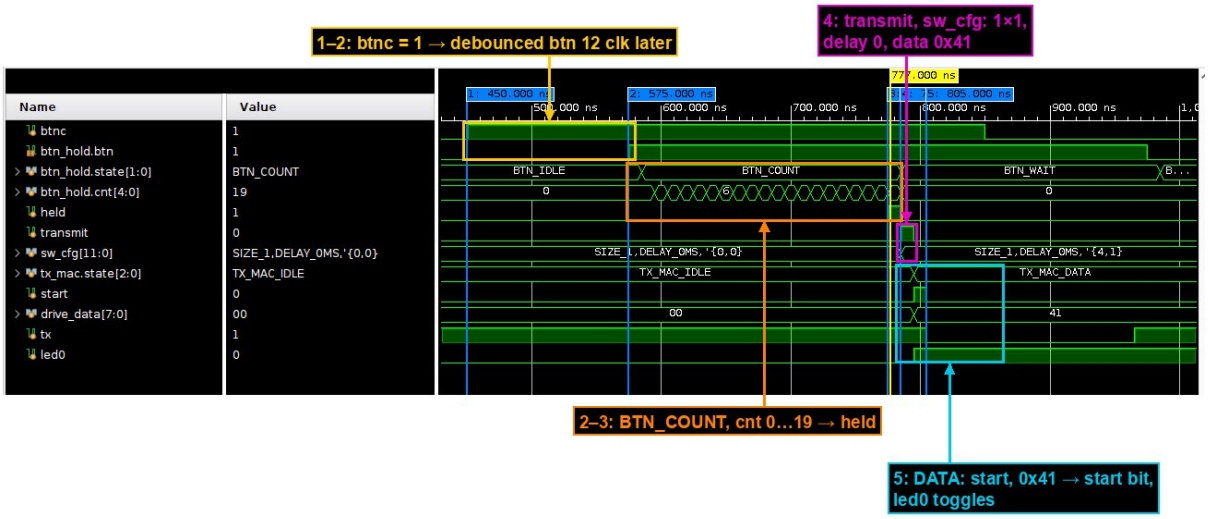
הריצה הסתיימה ב-860ns בלי ERROR בלוג, כלומר כל ה-assertions עברו. צורות הגל מוצגות באיור 28.



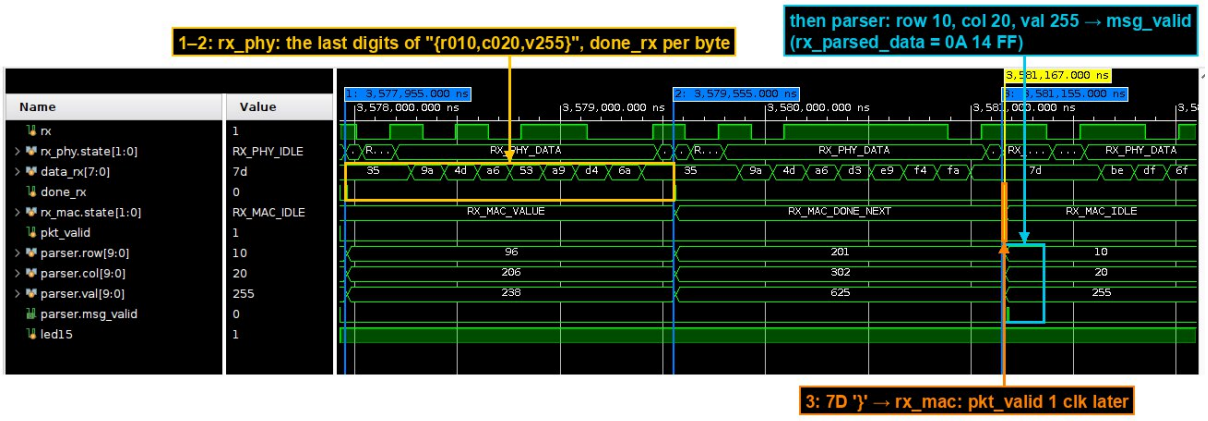
איור 28: tb_display_ctrl – כל הריצה: איפוס וסריקה אחת של שמונה הספרות.

4.15. tb_chip_top – בדיקת המערכת המלאה

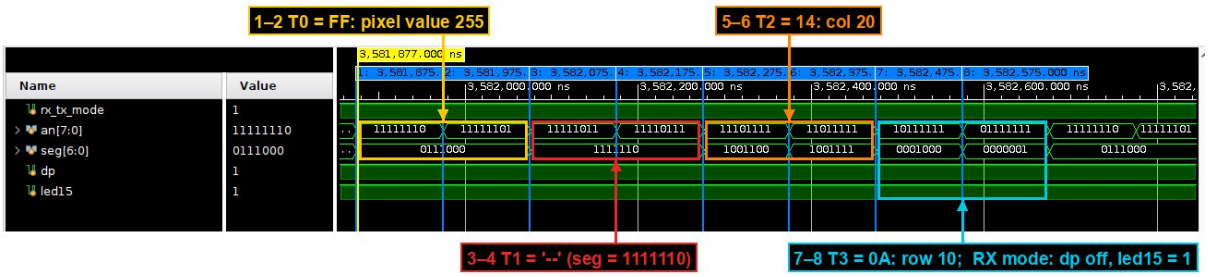
בטסטבנץ' זה אין assertions, ולכן התוצאה נבדקת בצורות הגל באיורים 32–29, והיא כצפוי. הריצה הסתיימה ב-3.61ms.



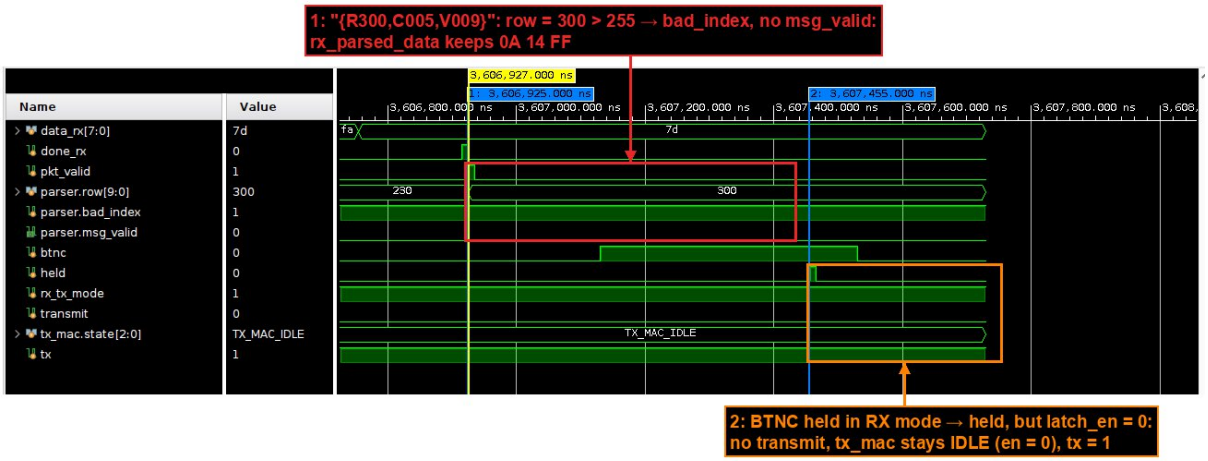
איור 29: tb_chip_top – זום: מהלחיצה על BTNC ועד ה-start bit.



איור 30: tb_chip_top – זום: סוף ההודעה {r010,c020,v255} והפענוח שלה.



איור 31: tb_chip_top – זום: סריקת התצוגה במצב RX.



איור 32: tb_chip_top – זום: ההודעה {R300,C005,V009} ולחיצה על BTNC במצב RX.

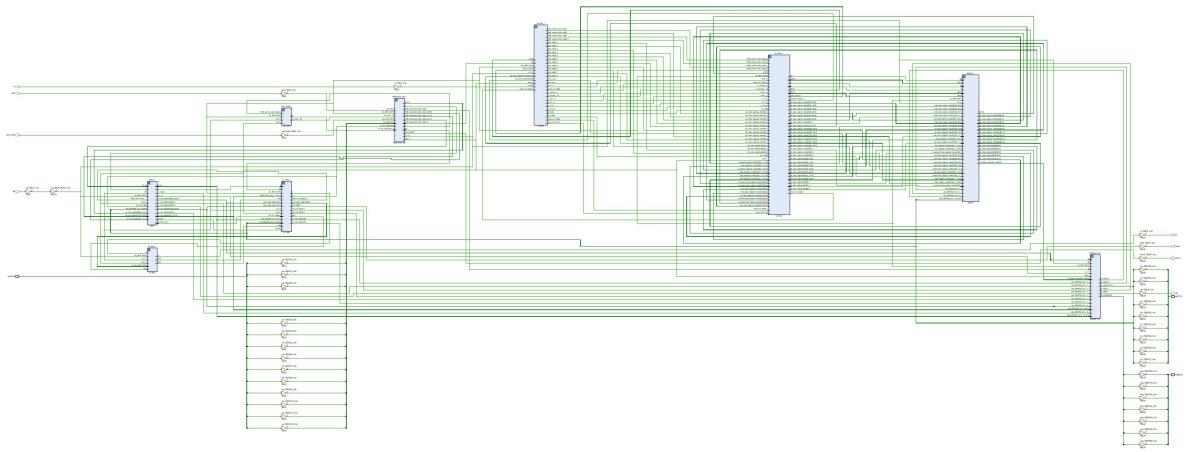
5. סינתזה ומימוש (Synthesis & Implementation)

5.1. סינתזה

טבלה 9: תוצאות הסינתזה ודוחות ה-Synthesized Design.

Report	Result
Messages (synthesis.log)	0 errors, 0 critical warnings, 52 warnings – all [Synth 8-7129]: parser inputs pkt_data[…] without a load
Timing, worst case (timing_summary_synth.rpt)	100 MHz: setup WNS +2.131 ns, hold WHS +0.079 ns – met
DRC (drc_synth.rpt)	0 checks
Methodology (methodology_synth.rpt)	0 checks
Clock networks / interaction	one clock: sys_clk_pin 100 MHz on clk (1 BUFG); only sys_clk_pin → sys_clk_pin paths
Noise (noise_synth.rpt)	19 output pins PASS, 0 FAIL
Power (power_synth.rpt)	0.108 W total on-chip (estimate)

כל 52 האזהרות של הסינתזה (טבלה 9) עוסקות בביטים של pkt_data שה-parser אינו קורא: הבתים { ו-} וה-nibble העליון של כל ספרה (סעיף 2.4). אלה כניסות של ה-parser בלי עומס, והשפעתן על מספר הדלגלגים מוסברת בפרק 6.



איור 33: סכמת ה-post-synthesis של chip_top.

5.2. מימוש (Implementation)

טבלה 10: תוצאות המימוש ודוחות ה-Implemented Design.

Report	Result
Messages (implementation.log)	0 errors, 0 critical warnings, 0 warnings
Timing, worst case (timing_summary_routed.rpt)	setup WNS +2.127 ns on rx_mac_i → parser_i (7.629 ns, 8 logic levels); hold WHS +0.099 ns – met
Route status (route_status.rpt)	2074 logical nets, 1140 routable, 1140 fully routed, 0 with routing errors
DRC (drc_routed.rpt)	0 checks
Methodology (methodology_drc_routed.rpt)	0 checks
Power (power_routed.rpt)	0.104 W total: 0.097 W static + 0.007 W dynamic; junction 25.5 °C
I/O (io_placed.rpt)	36 of 36 ports placed (FIXED) on the XDC pins, LVCMOS33 (sw[8], sw[9] LVCMOS18)

תוצאות המימוש מרוכזות בטבלה 10. סכמות ה-post-implementation תואמות את סכמת ה-post-synthesis, ולכן אינן מוצגות.

6. ניצול משאבים

הצפי למספר הדלגלגים נספר מה-RTL לפני הסינתזה, עם ערכי החומרה: בכל מודול נספרו כל הרגיסטרים לפי הרוחב שלהם. רוחב מונה הוא $clog2 של מספר הערכים שהוא מקבל (משוואות 11 ,10 ,9 ו-13). מכונת מצבים תופסת את רוחב ה-enum שלה, וכל כניסה מסונכרנת תופסת שני דלגלגים. בטבלה 11 רגיסטר בלי רוחב הוא ביט אחד, והצפי מושווה לסינתזת ה-OOC של כל מודול ולתכנון המלא.

$$W_{baud} = \lceil log_2 1736 \rceil = 11,\ W_{tick} = \lceil log_2 108 \rceil = 7,\ W_{display} = \lceil log_2 25000 \rceil = 15\ bit \qquad (13)$$

טבלה 11: צפי מול בפועל: דלגלגים בכל תת-מודול של chip_top ובתכנון המלא.

Module	Registers (name × width)	Expected FF	Actual FF (OOC)	Δ
debounce_top	14 × debounce (cnt×21, io_meta, io_sync, state×2); reset_meta, reset_sync, reset_prev, reset_cnt×21, cpu_reset_d	375	375	0
btn_hold	cnt×27, state×2	29	30	+1
latch	sw_cfg×12, transmit	13	13	0
tx_mac	delay_cnt×25, row_cnt×9, col_cnt×8, drive_data×8, state×3, start, led0	55	58	+3
tx_phy	sh_reg×10, cnt×4, state; baud_gen: cnt×11	26	25	−1
rx_phy	tick_cnt×7, tick16_cnt×4, bit_cnt×3, data×8, start_samples×3, rx_meta, rx_sync, rx_prev, state×2, start_check, done	32	34	+2
rx_mac	pkt_data×128, value_cnt×2, group_cnt×2, state×2, pkt_valid	135	137	+2
parser	rx_parsed_data×24, msg_valid	25	25	0
display_ctrl	an×8; baud_gen: cnt×15	23	23	0
Sum of modules		713	720	+7
Full design (synth / placed)		713	687 / 695	−26 / −18

ההפרשים:

- מכונות המצבים של btn_hold, tx_mac ו-rx_mac מקודדות ב-one-hot (סעיף 2.3), דלגלג לכל מצב, ולכן יש בהן 3, 6 ו-4 דלגלגי מצב במקום 2, 3 ו-2.
- ב-tx_phy הביט sh_reg[9] הוא תמיד 1 (נטען ב-1 ומוזז פנימה 1), ולכן הסינתזה מסירה אותו ו-sh_reg נשאר עם 9 דלגלגים.
- ב-rx_phy רגיסטר המצב נשמר פעמיים, כ-FSM_sequential_state_reg וכ-state_reg, כל אחד של 2 ביטים.
- התכנון המלא בסינתזה: היציאה msg_valid של ה-parser אינה מחוברת ב-chip_top ולכן מוסרת. ב-rx_mac מוסרים 12 דלגלגים שאף לוגיקה אינה קוראת: הבית msg_start, הביט r[5], שרק בו R ו-r נבדלים, וביט 5 של שלוש ספרות השורה, שמגיע בהזזה רק אליו. בנוסף, 24 שלבים של רגיסטר ההזזה pkt_data עוברים ל-11 SRL16E (רגיסטר הזזה בתוך LUT) עם 4 דלגלגים נוספים, ולכן ב-rx_mac נשארים 105 דלגלגים במקום 137.
- אחרי המימוש נוספים 8 דלגלגים: opt_design משכפל כל ביט של an לעותק an_reg[i]_lopt_replica, שמזין רק את הפין שלו, והרגיסטר המקורי מזין את הלוגיקה הפנימית.

הצפי ל-IOB הוא 36, כמספר הפינים בקובץ ה-XDC (17 כניסות ו-19 יציאות), והצפי ל-BUFG הוא אחד, לשעון היחיד. שניהם התקבלו כצפוי (טבלה 12). ל-LUTs לא נקבע צפי מדויק: רובם צפויים ב-debounce_top, שבו 15 מונים של 21 ביטים והמשווים שלהם, ואכן יש בו יותר LUTs מבכל מודול אחר (טבלה 13). התכנון כולו תופס פחות מ-2% מה-LUTs ומהדלגלגים של ה-FPGA ונכנס בו בקלות.

הערכים בטבלה 12 לקוחים מדוחות ה-utilization של התכנון המלא (utilization_synth.rpt ו-utilization_placed.rpt), והערכים בטבלה 13 מסינתזה out-of-context של כל מודול בנפרד, עם ערכי החומרה.

טבלה 12: ניצול התכנון המלא אחרי הסינתזה ואחרי המימוש.

Site type	Synthesis	Placed	Available	Util % (placed)
Slice LUTs	675	667	63,400	1.05
of them LUT as memory (SRL)	–	11	19,000	0.06
Slice Registers (flip-flops)	687	695	126,800	0.55
Register as Latch	0	0	126,800	0.00
Slices (placed: 165 SLICEL + 122 SLICEM)	–	287	15,850	1.81
F7 / F8 Muxes	0	0	31,700	0.00
Block RAM Tile	0	0	135	0.00
DSPs	0	0	240	0.00
Bonded IOB (17 inputs + 19 outputs)	36	36	210	17.14
BUFGCTRL	1	1	32	3.13
Unique control sets	38	38	15,850	0.24

טבלה 13: ניצול כל תת-מודול של chip_top בסינתזה out-of-context (כולל תת-המודולים שבתוכו).

Module	Contains	LUTs	CARRY4
debounce_top	debounce ×14	430	90
btn_hold	–	13	7
latch	–	3	0
tx_mac	–	69	11
tx_phy	baud_gen	36	0
rx_phy	–	43	0
rx_mac	–	39	0
parser	–	57	24
display_ctrl	baud_gen, tx_display, rx_display, seven_segment	57	4
Sum of modules		747	136
Full design (synth / placed)		675 / 667	136 / 136

7. בדיקה על הלוח

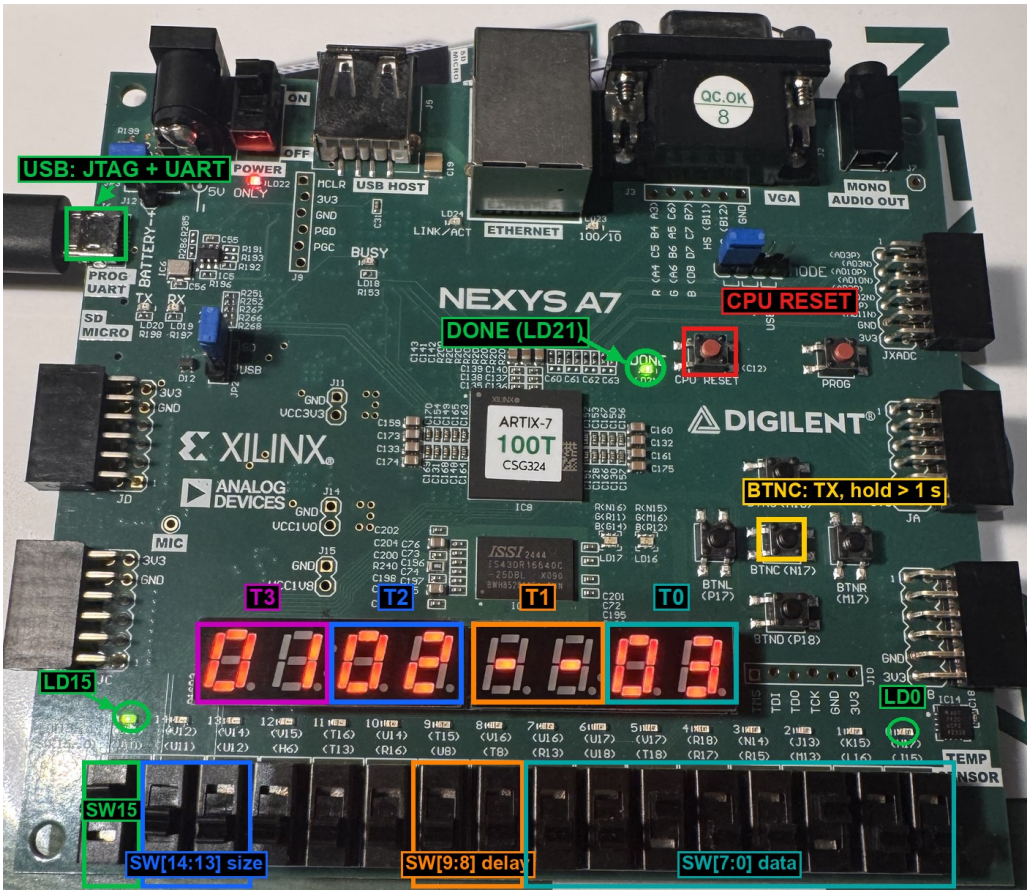
7.1. יצירת ה-bitstream וטעינתו ללוח

טבלה 14: סיכום יצירת ה-bitstream.

Item	Result
Bitstream	chip_top.bit – 0 errors, 0 critical warnings, 0 warnings
Device	xc7a100tcsg324-1 (Digilent Nexys A7-100T)
Clock constraint	create_clock sys_clk_pin, period 10.000 ns on clk (E3)
Programming	JTAG over the on-board USB port; DONE LED on
Terminal	TX: tio, log to output.txt; RX: CoolTerm. 57600 bps, 8N1, no flow control (USB-UART of the board)

7.2. בדיקה פונקציונלית על הלוח

נתיב ה-TX לא השתנה מאז מעבדה 4, ולכן מקרי ה-TX ותמונותיהם לקוחים מבדיקת הלוח של מעבדה 4.
מקרי ה-RX נבדקו על ה-bitstream של מעבדה זו, ואיור 34 מציג את הלוח במהלך בדיקת ה-RX.



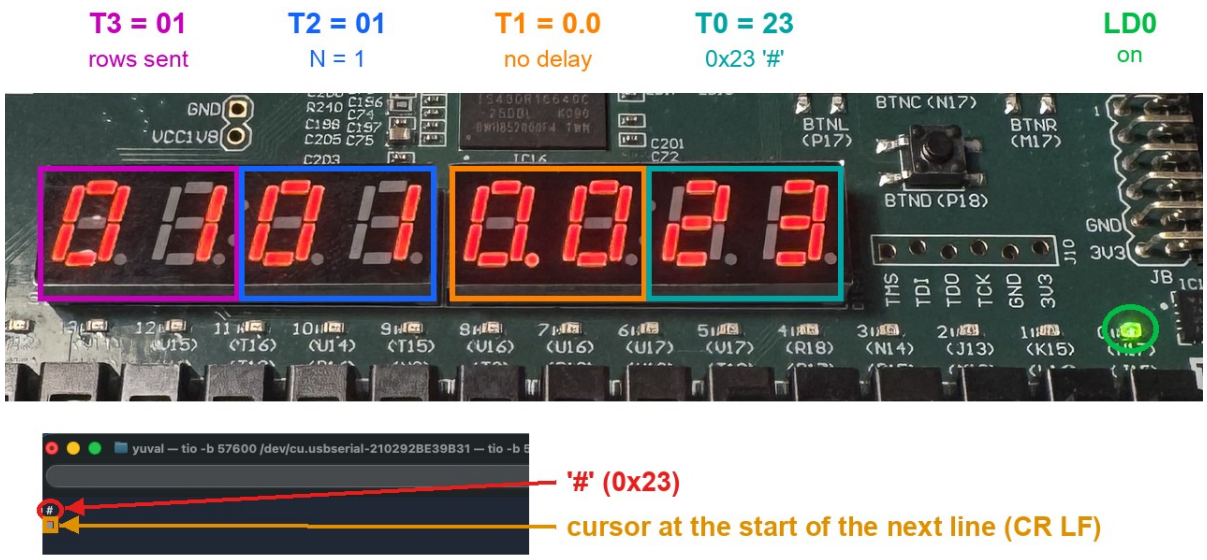
איור 34: הלוח לאחר טעינת ה-bitstream: נורית DONE דולקת, ו-LD15 דולק כי SW15=1 (מצב RX).
בתצוגה ההודעה של מקרה 5.

בבדיקת ה-TX הטרמינל הופעל עם tio, שרושם כל תו שמתקבל לקובץ output.txt:

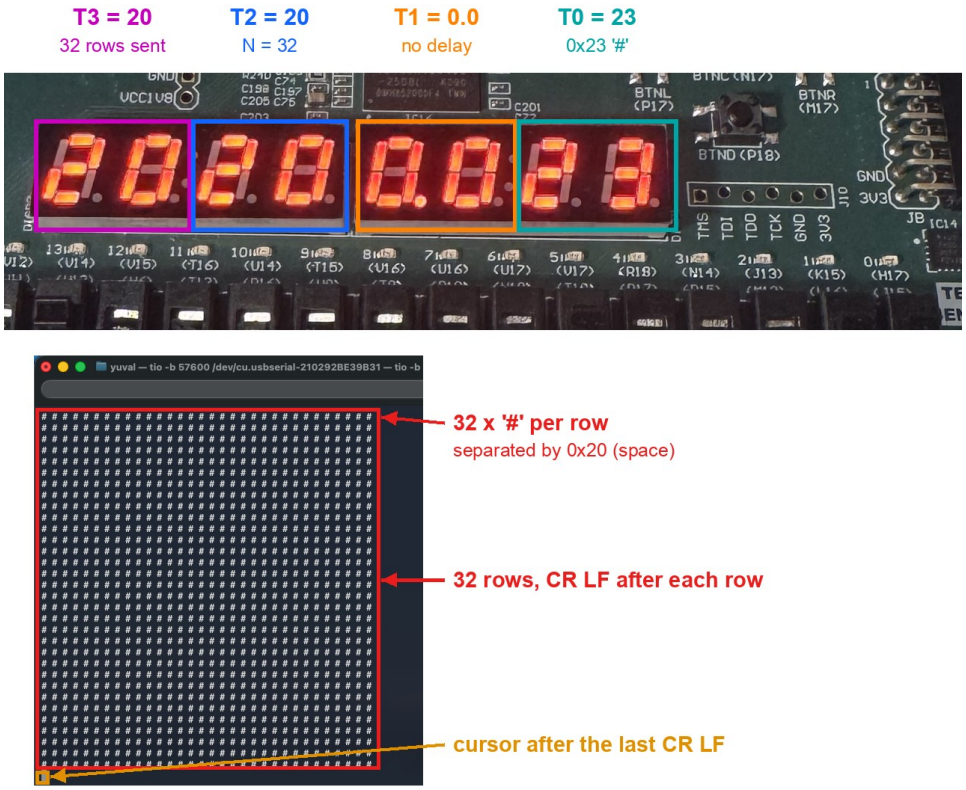
```
tio -b 57600 --log --log-file "$PWD/output.txt" --log-strip /dev/cu.usbserial-210292BE39B31
```

טבלה 15: מקרי הבדיקה על הלוח.

Case	SW	Button	Mode	Display	Result	Figure
1	0 / 00 / 00 / 0x23	BTNC > 1 s	TX, 1×1, 0 ms	01 01 0.0 23	Pass	35
2	0 / 01 / 00 / 0x23	BTNC > 1 s	TX, 32×32, 0 ms	20 20 0.0 23	Pass	36
3	0 / 11 / 00 / 0x23	BTNC > 1 s	TX, 256×256, 0 ms	29 FF 0.0 23 → FF FF 0.0 23	Pass	37
4	0 / 01 / 11 / 0x23	BTNC > 1 s	TX, 32×32, 200 ms	00 20 2.0 23 → 01 20 2.0 23	Pass	38
5	1 / – / – / –	–	RX, {R001,C002,V003}	01 02 -- 03	Pass	39
6	1 / – / – / –	–	RX, {r004,c005,v006}	04 05 -- 06	Pass	40
7	1 / – / – / –	–	RX, {R255,c255,V255}	FF FF -- FF	Pass	41
8	1 / – / – / –	–	RX, {a,b,c001}	FF FF -- FF (kept)	Pass	42
9	1 / – / – / –	–	RX, {R004,{R005,C020,V100}	05 14 -- 64	Pass	43
10	1 / – / – / –	–	RX, {R256,C256,V256}	05 14 -- 64 (kept)	Pass	44



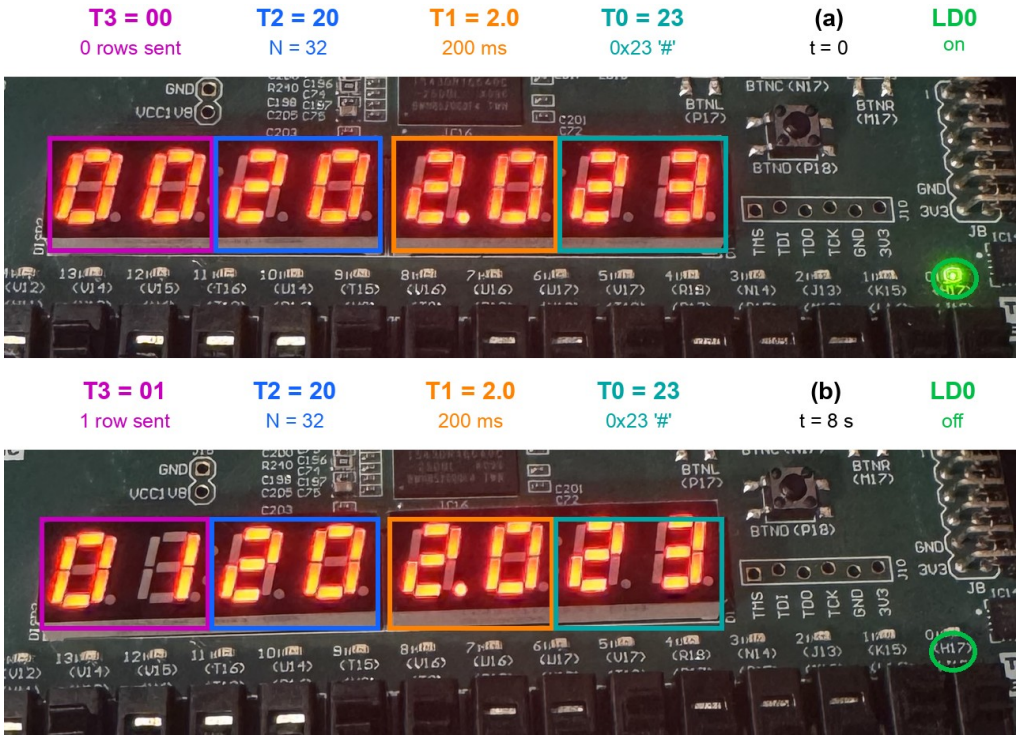
איור 35: מקרה TX, 1, ריבוע 1×1: התצוגה ו-LD0 (למעלה) והטרמינל (למטה).



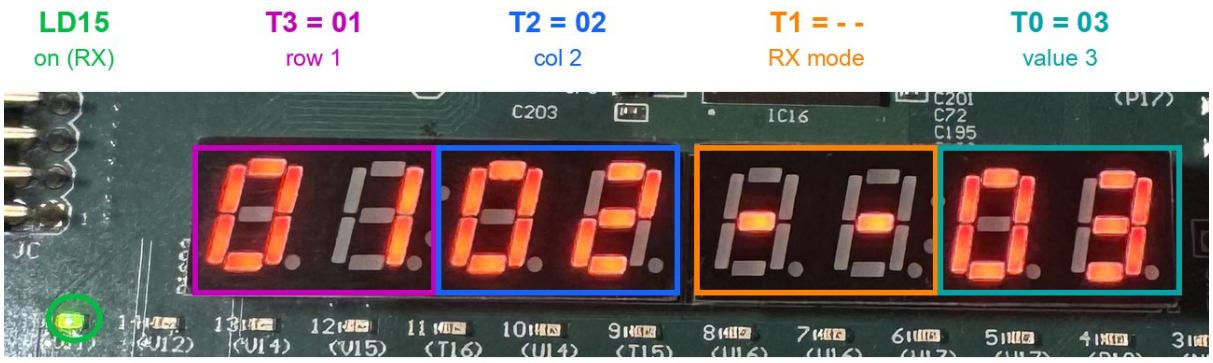
איור 36: מקרה TX, 2, ריבוע 32×32: התצוגה (למעלה) והטרמינל (למטה).

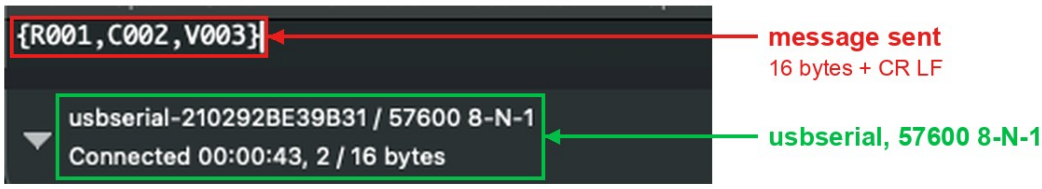


איור 37: מקרה TX, 3, ריבוע 256×256: (a) באמצע הריצה, (b) בסופה.



איור 38: מקרה TX, 4, ריבוע 32×32 עם השהיה של 200ms; תמונה (b) צולמה 8 שניות אחרי (a).

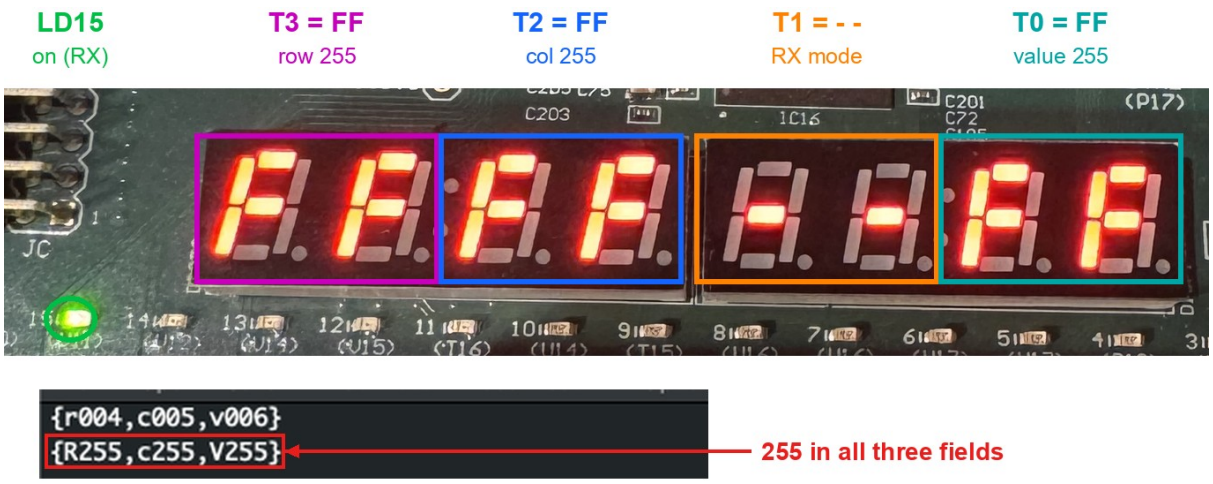




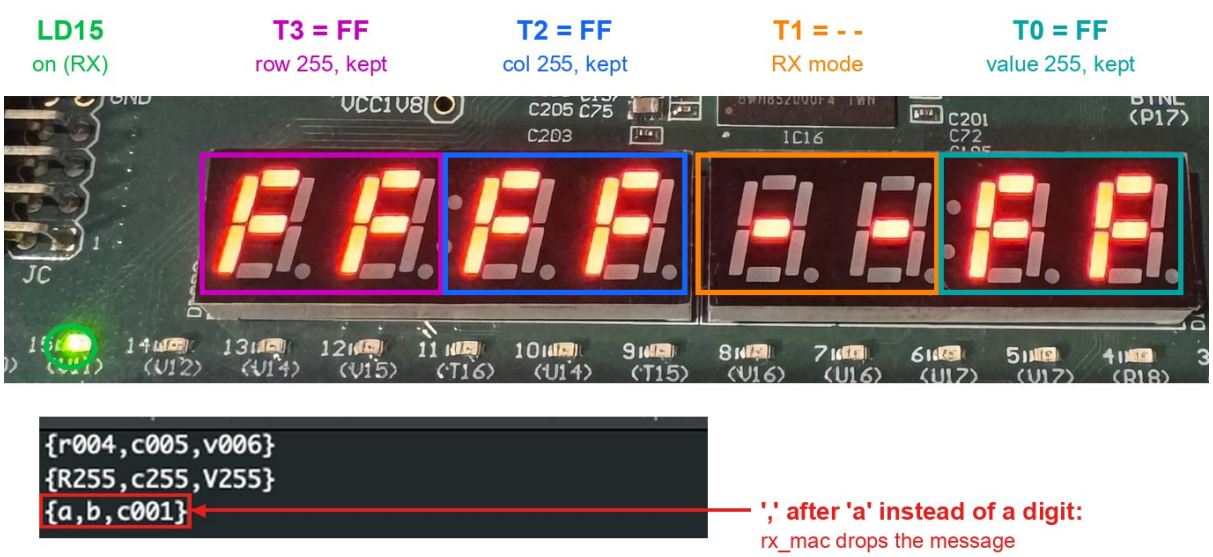
איור 39: מקרה RX, 5, ההודעה {R001,C002,V003}: התצוגה ו-LD15 (למעלה) וחלון CoolTerm (למטה).



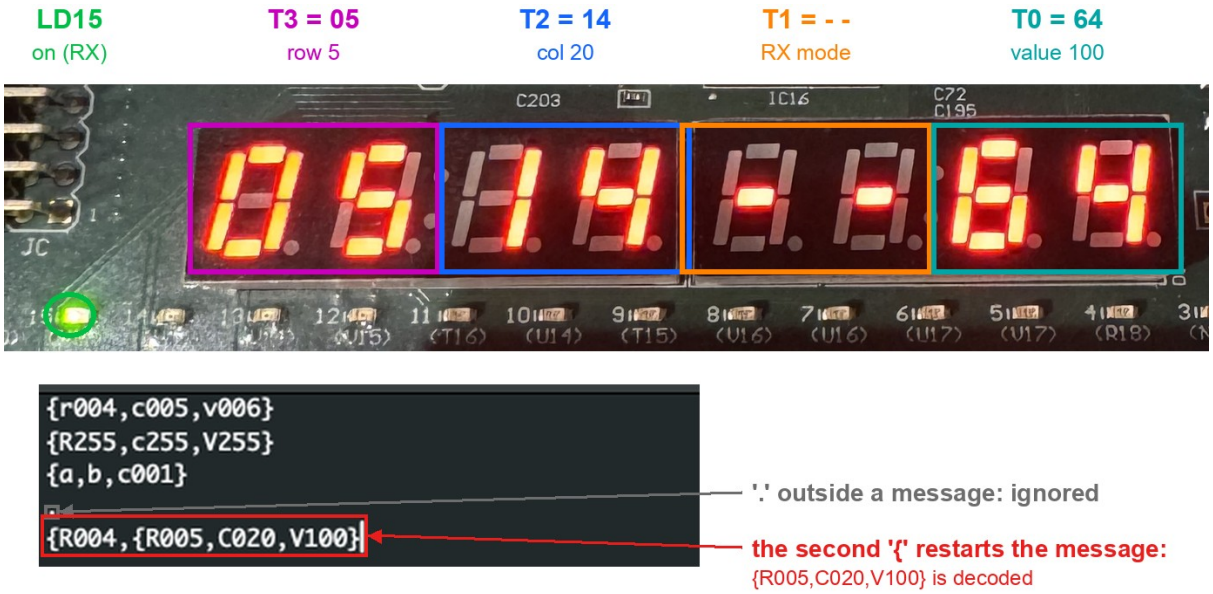
איור 40: מקרה RX ,6, אותיות קטנות: ההודעה {r004,c005,v006}.



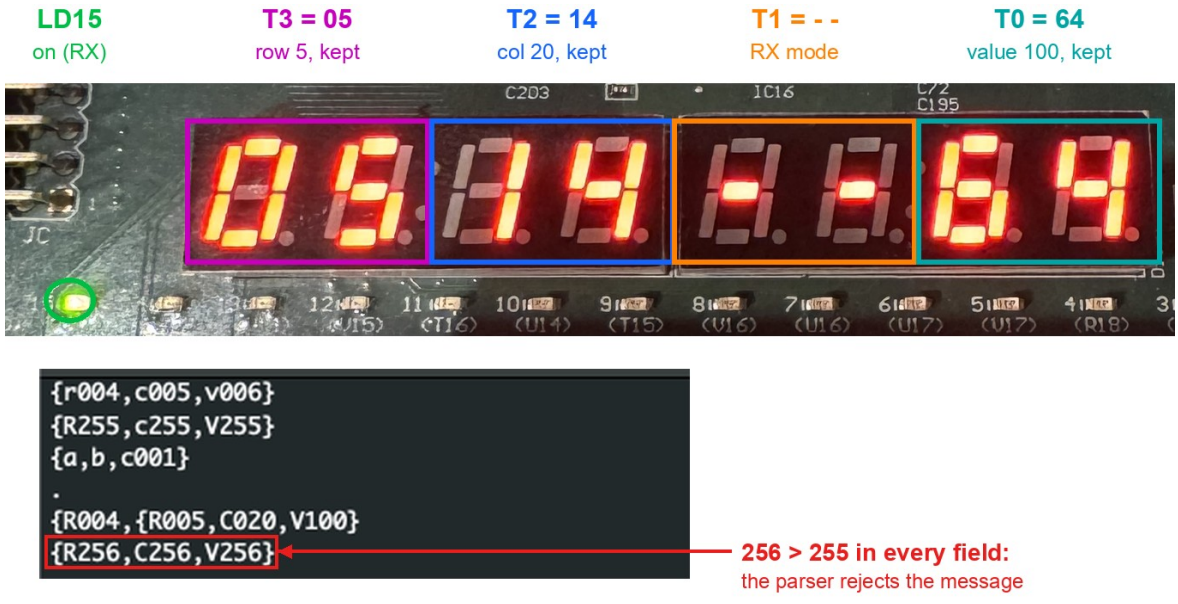
איור 41: מקרה RX ,7, הערך המרבי 255 בשלושת השדות: ההודעה {R255,c255,V255}.



איור 42: מקרה RX ,8, הודעה במבנה שגוי: ההודעה נדחית והתצוגה אינה משתנה.



איור 43: מקרה RX, 9, תו פתיחה שני באמצע ההודעה: הקליטה מתחילה מחדש.



איור 44: מקרה RX, 10, ערכים גדולים מ-255: ההודעה נדחית והתצוגה אינה משתנה.

8. החלטות תכנון, באגים ולקחים

8.1. החלטות תכנון

- קצב הסימולציה: BAUD_RATE=6.25MHz הוא CLK_FREQ/16, הקצב המהיר ביותר שבו ה-oversampling של 16 עדיין עובד. בערך הזה TICK_CNT של rx_phy הוא 1, כלומר tick בכל מחזור, וביט נמשך 16 מחזורים בדיוק גם במשדר וגם במקלט.
- פורטים של chip_top: וקטורים רגילים ולא struct. עם פורט struct, הסינתזה שיטחה את שמות השדות, ה-get_ports בקובץ ה-XDC לא מצא אף פורט והזרימה נכשלה.
- המרה מ-ASCII: ספרת ASCII היא 0x30–0x39, ולכן ערך הספרה הוא ה-nibble התחתון, בלי חיסור ובלי לוגיקה. שלוש הספרות מומרות לבינארי ברוחב ביניים של 10 ביטים (משוואה 7).
- רוחב מונים: מונה שמגיע ל-MAX עצמו דורש $clog2(MAX+1)$ ביטים. $clog2(MAX)$ מספיק למונה שעוצר ב-MAX−1, אבל כש-MAX חזקה של 2 הוא קצר בביט אחד ל-MAX עצמו. לכן row_out, שמגיע ל-256, ברוחב $clog2(256)+1=9$ ביטים, והמונה של btn_hold, שעוצר ב-BTN_HOLD_COUNT−1, ברוחב $clog2(BTN_HOLD_COUNT)$.
- msg_valid: יציאה זו של ה-parser אינה מחוברת ב-chip_top, בכוונה. היא קיימת עבור הטסטבנץ' וה-assertions.
- צמצום המעבר DONE_NEXT → IDLE ב-rx_mac: התנאי המקורי ("}", תו אחר, או פסיק כש-max_group=1, אבל לא "{") פושט באלגברה ובמפת קרנו. is_start, is_comma ו-is_end מוציאים זה את זה, ולכן צירופים שבהם יותר מאחד מהם 1 הם don't cares, ו-is_end נשמט מהביטוי (משוואה 14). במהלך הצמצום תא אחד במפה סומן במקום שגוי ותוקן.

$$next = IDLE \;\Leftrightarrow\; done\cdot[(\neg is_comma\cdot\neg is_start) + (max_group\cdot is_comma)] \tag{14}$$

8.2. באגים שנמצאו ותוקנו

- תזמון pkt_valid: הוא נקבע מהבית הנכנס (DONE_NEXT, done ו-"}") ונרשם ברגיסטר. אילו נגזר מהמצב שאחרי המעבר, היה עולה רק עם הבית הבא, בית אחד מאוחר מדי.
- BTNC במצב RX שלח בית (על הלוח): במצב RX (SW15=1), החזקה של BTNC שנייה שלחה בית אחד ב-tx, ו-LED0 התהפך. הסיבה: ל-latch לא היה תנאי מצב. latch_en היה ready & held, ו-ready של (tx_mac (state == TX_MAC_IDLE הוא 1 גם במצב RX, ולכן התצורה ננעלה ו-transmit עלה. ה-en של tx_mac חסם רק את המצב הבא, ולא את data_load ואת start, ולכן פולס start אחד הגיע ל-tx_phy ופריים אחד יצא, בזמן שמכונת המצבים עצמה נשארה ב-TX_MAC_IDLE. התיקון: latch_en = ready & held & !rx_tx_mode. במצב RX לחיצה על BTNC אינה נועלת תצורה ואינה מפעילה transmit, ותצוגת ה-TX ממשיכה להראות את התצורה האחרונה ששודרה.
- col_cnt אחרי ריצה שנעצרה (ב-tb_tx_mac): ריצה שנעצרה ב-en=0 השאירה את col_cnt בערכו, והריצה הבאה התחילה בעמודה שגויה. התיקון: col_cnt מתאפס גם ב-TX_MAC_IDLE & transmit.

9. סיכום

במעבדה מומש ב-SystemVerilog בקר UART דו-כיווני ללוח Nexys A7-100T: נתיב TX שמשדר ל-PC ריבוע של בית לפי תצורת המתגים, ונתיב RX שמפענח הודעות {Rddd,Cddd,Vddd} ומציג אותן בתצוגת ה-Seven Segment. כל 15 הטסטבנצ'ים רצו בלי ERROR בלוג, ובטסטבנצ'ים שאין בהם assertions התוצאה נבדקה בצורות הגל. הצפי מה-RTL היה 713 דלגלגים, ובפועל התקבלו 687 בסינתזה ו-695 אחרי המימוש, וכל הפרש מוסבר בפרק 6. התזמון עומד בשעון של 100MHz, עם WNS של 2.127ns אחרי המימוש. על הלוח עברו כל מקרי הבדיקה: מקרי ה-RX על ה-bitstream של מעבדה זו, ומקרי ה-TX ממעבדה 4, כי נתיב ה-TX לא השתנה מאז. הלקח העיקרי עלה בבדיקה על הלוח: ה-en של tx_mac חסם רק את המצב הבא ולא את start, ולכן נוסף ל-latch התנאי rx_tx_mode! (סעיף 8.2).

נספח א': קוד המקור (RTL ואילוצים)

pkg/lab5_pkg.sv

```
package lab5_pkg;

   // Global
   // Clock frequency
   localparam CLK_FREQ = 100000000;
   localparam DATA_WIDTH = 8;
   localparam DIG_IN_WIDTH = 4;

   localparam PACKET_SIZE = 16;

   // TB Values
   localparam CLK_PERIOD = 10;

   // UART frame (tx_phy)
   localparam START_BIT_SIZE = 1;
   localparam STOP_BIT_SIZE = 1;
   localparam FRAME_WIDTH = START_BIT_SIZE + DATA_WIDTH + STOP_BIT_SIZE;

   // Switch configuration (debounce_top -> latch -> tx_mac, display_ctrl)
   localparam SIZE_SEL_WIDTH = 2;
   localparam DELAY_SEL_WIDTH = 2;
   localparam MODE_WIDTH = 1;
   localparam SW_WIDTH = DATA_WIDTH + SIZE_SEL_WIDTH + DELAY_SEL_WIDTH + MODE_WIDTH;

   localparam [1:0][DIG_IN_WIDTH-1:0] DELAY_0MS_DISP = 8'h00, DELAY_50MS_DISP = 8'h05,
DELAY_100MS_DISP = 8'h10, DELAY_200MS_DISP = 8'h20;

   localparam SIZE_1_VAL = 1, SIZE_32_VAL = 32, SIZE_128_VAL=128, SIZE_256_VAL = 256;
   typedef enum logic [SIZE_SEL_WIDTH-1:0] { SIZE_1, SIZE_32, SIZE_128, SIZE_256 } size_cfg_t;
   typedef enum logic [DELAY_SEL_WIDTH-1:0] { DELAY_0MS, DELAY_50MS, DELAY_100MS, DELAY_200MS }
delay_cfg_t;

   typedef struct packed {
      size_cfg_t size; //sw[11:10] -> switches 14-13
      delay_cfg_t delay; //sw[9:8] -> switches 9-8
      logic [1:0][DIG_IN_WIDTH-1:0] data; //sw[7:0] -> switches 7-0
   } sw_cfg_t;

   // For extra switch in chipt_top -> debounce -> latch flow
   typedef struct packed {
      logic mode;
      logic btnc;
      sw_cfg_t cfg;
   } board_in_t;

   // TX path (tx_mac, tx_display)
   localparam SIZE_WIDTH = $clog2(SIZE_256_VAL);
   localparam ROW_SIZE_WIDTH = SIZE_WIDTH + 1;

   localparam [DATA_WIDTH-1:0] SPACE_DATA = 8'h20;
   localparam [DATA_WIDTH-1:0] NEW_LINE_0_DATA = 8'h0D;
   localparam [DATA_WIDTH-1:0] NEW_LINE_1_DATA = 8'h0A;
```

```
// RX message (rx_mac -> parser)
localparam DIGITS = 3;
localparam MAX_GROUPS = 3;
localparam NUM_OF_VALUES = 3;
typedef struct packed {
    logic [DATA_WIDTH-1:0] msg_start;
    logic [DATA_WIDTH-1:0] r;
    logic [DIGITS-1:0][DATA_WIDTH-1:0] row_index;
    logic [DATA_WIDTH-1:0] comma0;
    logic [DATA_WIDTH-1:0] c;
    logic [DIGITS-1:0][DATA_WIDTH-1:0] col_index;
    logic [DATA_WIDTH-1:0] comma1;
    logic [DATA_WIDTH-1:0] v;
    logic [DIGITS-1:0][DATA_WIDTH-1:0] value;
    logic [DATA_WIDTH-1:0] msg_close;
} packet_cfg_t;

localparam [DATA_WIDTH-1:0] MSG_START_VAL = 8'h7B;
localparam [DATA_WIDTH-1:0] R_VAL = 8'h52;
localparam [DATA_WIDTH-1:0] R_SMALL_VAL = 8'h72;
localparam [DATA_WIDTH-1:0] COMMA_VAL = 8'h2C;
localparam [DATA_WIDTH-1:0] C_VAL = 8'h43;
localparam [DATA_WIDTH-1:0] C_SMALL_VAL = 8'h63;
localparam [DATA_WIDTH-1:0] V_VAL = 8'h56;
localparam [DATA_WIDTH-1:0] V_SMALL_VAL = 8'h76;
localparam [DATA_WIDTH-1:0] MSG_CLOSE_VAL = 8'h7D;

// RX result (parser -> display_ctrl -> rx_display)
localparam SCREENS_PER_SEGMENT = 2;
typedef struct packed {
    logic [SCREENS_PER_SEGMENT-1:0][DIG_IN_WIDTH-1:0] row_index;
    logic [SCREENS_PER_SEGMENT-1:0][DIG_IN_WIDTH-1:0] col_index;
    logic [SCREENS_PER_SEGMENT-1:0][DIG_IN_WIDTH-1:0] pixel_value;
} rx_parsed_t;

// Display (rx_display, tx_display -> seven_segment -> display_ctrl -> pins)
// Number of anodes for screen
localparam AN_WIDTH = 8;
localparam SS_DIG_WIDTH = 7;
localparam [SS_DIG_WIDTH-1:0] DASH_VAL = 7'b111_1110;
localparam [AN_WIDTH-1:0] AN_RESET_VAL = {{(AN_WIDTH-1){1'b1}}, 1'b0};
localparam [SS_DIG_WIDTH-1:0] SEG_RESET_VAL = 7'd1;
localparam DP_RESET_VAL = 1'b1;

typedef struct packed {
    logic dash;
    logic [DIG_IN_WIDTH-1:0] data_val;
    logic dp;
} seven_segment_data_t;

typedef struct packed {
    logic [SS_DIG_WIDTH-1:0] ss_dig;
    logic dp;
} display_ctrl_data_t;
```

```
    typedef struct packed {
        logic [AN_WIDTH-1:0] an;
        logic [SS_DIG_WIDTH-1:0] seg;
        logic dp;
    } display_cfg_t;

    // FSM states, in data-flow order
    typedef enum logic [1:0] { DEB_S0, DEB_W1, DEB_S1, DEB_W0 } deb_state_t;
    typedef enum logic [1:0] { BTN_IDLE, BTN_COUNT, BTN_WAIT } btn_state_t;
    typedef enum logic [2:0] { TX_MAC_IDLE, TX_MAC_DATA, TX_MAC_SPACE, TX_MAC_NEW_LINE_0,
TX_MAC_NEW_LINE_1, TX_MAC_DELAY } tx_mac_state_t;
    typedef enum logic { TX_PHY_IDLE, TX_PHY_TRANSMIT } tx_phy_state_t;
    typedef enum logic [1:0] { RX_PHY_IDLE, RX_PHY_START, RX_PHY_DATA, RX_PHY_STOP } rx_phy_state_t;
    typedef enum logic [1:0] { RX_MAC_IDLE, RX_MAC_OPCODE, RX_MAC_VALUE, RX_MAC_DONE_NEXT }
rx_mac_state_t;

endpackage
```

defines/lab5_definitions.svh

```
`ifndef LAB5_DEFINITIONS_SVH
`define LAB5_DEFINITIONS_SVH

`ifdef SIM
    `define BAUD_RATE 6250000
    `define DEB_COUNT 10
    `define BTN_HOLD_COUNT 20
    `define DELAY_50MS_VAL 10
    `define DISPLAY_RR 10000000
`else
    `define BAUD_RATE 57600
    `define DEB_COUNT 2000000
    `define BTN_HOLD_COUNT 100000000
    `define DELAY_50MS_VAL 5000000
    `define DISPLAY_RR 4000
`endif




`endif
```

src/chip_top.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module chip_top #(
    parameter BAUD_RATE = `BAUD_RATE,
    parameter DEB_COUNT = `DEB_COUNT,
    parameter BTN_HOLD_COUNT = `BTN_HOLD_COUNT,
    parameter DELAY_50MS_VAL = `DELAY_50MS_VAL,
```

```
    parameter DISPLAY_RR = `DISPLAY_RR
) (
    input logic clk, cpu_reset,
    input logic [SW_WIDTH-1:0] sw,
    input logic btnc,
    input logic rx,
    output logic tx,
    output logic [AN_WIDTH-1:0] an,
    output logic [SS_DIG_WIDTH-1:0] seg,
    output logic dp,
    output logic led0, led15
);

    board_in_t io_in; assign io_in = {sw[SW_WIDTH-1], btnc, sw[SW_WIDTH-2:0]};

    // debounce
    logic reset_n;
    board_in_t io_d;
    debounce_top #(.DEB_COUNT(DEB_COUNT)) debounce_top_i(
        .clk(clk),
        .cpu_reset(cpu_reset),
        .io_in(io_in),
        .io_d(io_d),
        .cpu_reset_d(reset_n)
    );

    logic rx_tx_mode; assign rx_tx_mode = io_d.mode;

    // btn_hold
    logic held;
    btn_hold #(.BTN_HOLD_COUNT(BTN_HOLD_COUNT)) btn_hold_i(
        .clk(clk),
        .reset_n(reset_n),
        .btn(io_d.btnc),
        .held(held)
    );


    // latch
    logic transmit, ready;
    sw_cfg_t sw_cfg;
    latch latch_i(
        .clk(clk),
        .reset_n(reset_n),
        .ready(ready),
        .sw_d(io_d.cfg),
        .held(held),
        .rx_tx_mode(rx_tx_mode),
        .sw_cfg(sw_cfg),
        .transmit(transmit)
    );

    //tx_mac
    logic done, start;
    logic [DATA_WIDTH-1:0] drive_data;
    logic [ROW_SIZE_WIDTH-1:0] row_out;
```

```
tx_mac #(.DELAY_50MS_VAL(DELAY_50MS_VAL)) tx_mac_i(
    .clk(clk),
    .reset_n(reset_n),
    .en(!rx_tx_mode),
    .sw_d(sw_cfg),
    .transmit(transmit),
    .done(done),
    .drive_data(drive_data),
    .start(start),
    .led0(led0),
    .row_out(row_out),
    .ready(ready)
);

//tx_phy
tx_phy #(.BAUD_RATE(BAUD_RATE)) tx_phy_i(
    .clk(clk),
    .reset_n(reset_n),
    .start(start),
    .data(drive_data),
    .tx(tx),
    .done(done)
);

//rx_phy
logic [DATA_WIDTH-1:0] data_rx;
logic done_rx;
rx_phy #(.BAUD_RATE(BAUD_RATE)) rx_phy_i(
    .clk(clk),
    .reset_n(reset_n),
    .en(rx_tx_mode),
    .rx(rx),
    .data(data_rx),
    .done(done_rx)
);

//rx_mac
packet_cfg_t pkt_data;
logic pkt_valid;
rx_mac rx_mac_i(
    .clk(clk),
    .reset_n(reset_n),
    .done(done_rx),
    .data_in(data_rx),
    .pkt_data(pkt_data),
    .pkt_valid(pkt_valid)
);

//parser
rx_parsed_t rx_parsed_data;
parser parser_i(
    .clk(clk),
    .reset_n(reset_n),
    .pkt_data(pkt_data),
    .pkt_valid(pkt_valid),
    .rx_parsed_data(rx_parsed_data)
```

```
    );

    //display_ctrl
    display_cfg_t disp_cfg;
    display_ctrl #(.DISPLAY_RR(DISPLAY_RR)) display_ctrl_i(
        .clk(clk),
        .reset_n(reset_n),
        .rx_data(rx_parsed_data),
        .tx_data(sw_cfg),
        .row_out(row_out),
        .rx_tx_mode(rx_tx_mode),
        .display_out(disp_cfg)
    );
    assign {an, seg, dp} = disp_cfg;
    assign led15 = rx_tx_mode;


endmodule
```

src/debounce_top.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module debounce_top
#(
    parameter DEB_COUNT = `DEB_COUNT
) (
    input logic clk, cpu_reset,
    input board_in_t io_in,
    output board_in_t io_d,
    output logic cpu_reset_d = '0
);

    localparam DEB_COUNT_WIDTH = $clog2(DEB_COUNT);
    localparam [DEB_COUNT_WIDTH-1:0] ONE = 1;
    localparam BTN_C_WIDTH = 1;
    localparam IO_WIDTH = SW_WIDTH + BTN_C_WIDTH;

    // Reset Debounce
    // Initial value for reset regs are 0, so that dut has power on reset
    logic reset_sync = '0;
    logic reset_meta = '0;
    always_ff @(posedge clk) begin
        {reset_sync, reset_meta} <= {reset_meta, cpu_reset};
    end

    logic reset_prev = '0;
    always_ff @(posedge clk) begin
        reset_prev <= reset_sync;
    end
    logic reset_edge;
    assign reset_edge = reset_prev ^ reset_sync;

    logic [DEB_COUNT_WIDTH-1:0] reset_cnt = '0;
```

```
  logic reset_stable;
  assign reset_stable = (reset_cnt == DEB_COUNT - 1);
  always_ff @(posedge clk) begin
    if (reset_edge) reset_cnt <= '0;
    else if (!reset_stable) reset_cnt <= reset_cnt + ONE;
  end

  always_ff @(posedge clk) begin
    if (reset_stable & !reset_edge) cpu_reset_d <= reset_sync;
  end


  // SW + BTNC Debounce

  genvar i;
  generate
    for(i = 0; i < IO_WIDTH; i++) begin : io_deb
      if (i == IO_WIDTH - 1) debounce #(.DEB_COUNT(DEB_COUNT)) debounce_i (clk, cpu_reset_d,
io_in.mode, io_d.mode);
      else if (i == IO_WIDTH - 2) debounce #(.DEB_COUNT(DEB_COUNT)) debounce_i (clk, cpu_reset_d,
io_in.btnc, io_d.btnc);
      else debounce #(.DEB_COUNT(DEB_COUNT)) debounce_i (clk, cpu_reset_d, io_in.cfg[i], io_d.cfg[i]);
    end
  endgenerate


endmodule
```

src/debounce.sv

```
import lab5_pkg::*;

module debounce #(
  parameter DEB_COUNT = 2000000
) (
  input logic clk, reset_n, io_in,
  output logic io_d
);

  localparam COUNT_WIDTH = DEB_COUNT > 1 ? $clog2(DEB_COUNT) : 1;
  localparam [COUNT_WIDTH-1:0] ONE = 1;

  logic [COUNT_WIDTH-1:0] cnt;

  logic io_meta, io_sync;
  always_ff @(posedge clk or negedge reset_n) begin
    if (!reset_n) {io_sync, io_meta} <= '0;
    else {io_sync, io_meta} <= {io_meta, io_in};
  end


  deb_state_t state, next;
  logic cnt_max; assign cnt_max = (cnt == DEB_COUNT-1);
  always_ff @(posedge clk or negedge reset_n) begin
```

```
    if (!reset_n) cnt <= '0;
    else if (state != next) cnt <= '0;
    else if ((state == DEB_W0) || (state == DEB_W1)) cnt <= cnt + ONE;
  end

  always_ff @(posedge clk or negedge reset_n) begin
    if (!reset_n) state <= DEB_S0;
    else state <= next;
  end

  always_comb begin
    next = state;
    unique case(state)
      DEB_S0: next = (io_sync) ? DEB_W1 : DEB_S0;
      DEB_W1: next = (!io_sync) ? DEB_S0 : ((cnt_max) ? DEB_S1 : DEB_W1);
      DEB_S1: next = (!io_sync) ? DEB_W0 : DEB_S1;
      DEB_W0: next = (io_sync) ? DEB_S1 : ((cnt_max) ? DEB_S0 : DEB_W0);
      default: next = state;
    endcase
  end

  assign io_d = (state == DEB_S1) || (state == DEB_W0);

endmodule
```

src/btn_hold.sv

```
`include "lab5_definitions.svh"
module btn_hold #(parameter BTN_HOLD_COUNT = `BTN_HOLD_COUNT) (
  input logic clk, reset_n, btn,
  output logic held
);
  import lab5_pkg::*;

  localparam BTN_COUNT_WIDTH = $clog2(BTN_HOLD_COUNT);
  localparam [BTN_COUNT_WIDTH-1:0] ONE = 1;
  logic [BTN_COUNT_WIDTH-1:0] cnt;
  logic cnt_max;
  assign cnt_max = cnt == BTN_HOLD_COUNT - 1;
  btn_state_t state, next;

  always_ff @(posedge clk) begin
    if (!reset_n) state <= BTN_IDLE;
    else state <= next;
  end

  always_comb begin
    next = state;
    unique case(state)
      BTN_IDLE: next = btn ? BTN_COUNT : BTN_IDLE;
      BTN_COUNT: next = (!btn) ? BTN_IDLE : (cnt_max ? BTN_WAIT : BTN_COUNT);
      BTN_WAIT: next = btn ? BTN_WAIT : BTN_IDLE;
      default: next = state;
    endcase
  end
```

```
    always_ff @(posedge clk) begin
        if (!reset_n) cnt <= '0;
        else if (state != next) cnt <= '0;
        else if (state == BTN_COUNT) cnt <= cnt + ONE;
    end

    assign held = (state == BTN_COUNT) & btn & cnt_max;



endmodule
```

src/latch.sv

```
import lab5_pkg::*;

module latch (
    input logic clk, reset_n, ready,
    input sw_cfg_t sw_d,
    input logic held,
    input logic rx_tx_mode,
    output sw_cfg_t sw_cfg,
    output logic transmit
);

    logic latch_en;
    assign latch_en = ready & held & !rx_tx_mode;

    always_ff @(posedge clk) begin
        if (!reset_n) {transmit, sw_cfg} <= '0;
        else if (latch_en) {transmit, sw_cfg} <= {1'b1, sw_d};
        else transmit <= '0;
    end

endmodule
```

src/tx_mac.sv

```
`include "lab5_definitions.svh"
import lab5_pkg::*;

module tx_mac
# (
    parameter DELAY_50MS_VAL = `DELAY_50MS_VAL
) (
    input logic clk, reset_n,
    input logic en,
    input sw_cfg_t sw_d,
    input logic transmit,
    input logic done,
    output logic [DATA_WIDTH-1:0] drive_data,
    output logic start, led0,
```

```
    output logic [ROW_SIZE_WIDTH-1:0] row_out,
    output logic ready
);

    localparam DELAY_0MS_VAL = 1, DELAY_100MS_VAL = 2 * DELAY_50MS_VAL, DELAY_200MS_VAL = 4 *
DELAY_50MS_VAL;
    localparam DELAY_WIDTH = $clog2(DELAY_200MS_VAL);

    localparam COL_SIZE_WIDTH = SIZE_WIDTH;

    localparam [DELAY_WIDTH-1:0] DELAY_ONE = 1;
    localparam [COL_SIZE_WIDTH-1:0] COL_SIZE_ONE = 1;
    localparam [ROW_SIZE_WIDTH-1:0] ROW_SIZE_ONE = 1;

    tx_mac_state_t state, next;

    logic [DELAY_WIDTH-1:0] delay_reg;
    always_comb begin
        unique case (sw_d.delay)
            DELAY_0MS: delay_reg = DELAY_0MS_VAL - 1;
            DELAY_50MS: delay_reg = DELAY_50MS_VAL - 1;
            DELAY_100MS: delay_reg = DELAY_100MS_VAL - 1;
            DELAY_200MS: delay_reg = DELAY_200MS_VAL - 1;
            default: delay_reg = DELAY_0MS_VAL - 1;
        endcase
    end

    logic [SIZE_WIDTH-1:0] size_reg;
    always_comb begin
        unique case(sw_d.size)
            SIZE_1: size_reg = SIZE_1_VAL - 1;
            SIZE_32: size_reg = SIZE_32_VAL - 1;
            SIZE_128: size_reg = SIZE_128_VAL - 1;
            SIZE_256: size_reg = SIZE_256_VAL - 1;
            default: size_reg = SIZE_1_VAL - 1;
        endcase
    end

    logic [COL_SIZE_WIDTH-1:0] col_cnt;
    logic last_col;
    assign last_col = (col_cnt == size_reg);
    logic col_cnt_reset;
    assign col_cnt_reset = ((state == TX_MAC_NEW_LINE_1) & done) || ((state == TX_MAC_IDLE) & transmit); //
New reset condition (found bug in)
    logic col_cnt_en;
    assign col_cnt_en = (state == TX_MAC_SPACE) & done;
    always_ff @(posedge clk) begin
        if (!reset_n) col_cnt <= '0;
        else if (col_cnt_reset) col_cnt <= '0;
        else if (col_cnt_en) col_cnt <= col_cnt + COL_SIZE_ONE;
    end

    logic [ROW_SIZE_WIDTH-1:0] row_cnt;
    logic last_row;
    assign last_row = (row_cnt == {1'b0, size_reg});
    logic row_cnt_reset;
```

```
  assign row_cnt_reset = (state == TX_MAC_IDLE) & transmit;
  logic row_cnt_en;
  assign row_cnt_en = (state == TX_MAC_NEW_LINE_1) & done;
  always_ff @(posedge clk) begin
    if (!reset_n) row_cnt <= '0;
    else if (row_cnt_reset) row_cnt <= '0;
    else if (row_cnt_en) row_cnt <= row_cnt + ROW_SIZE_ONE;
  end


  logic [DELAY_WIDTH-1:0] delay_cnt;
  logic delay_done; assign delay_done = (delay_cnt == delay_reg);
  logic delay_cnt_reset; assign delay_cnt_reset = ((state == TX_MAC_SPACE) & done) | ((state ==
TX_MAC_NEW_LINE_1) & done & !last_row);
  always_ff @(posedge clk) begin
    if (!reset_n) delay_cnt <= '0;
    else if (delay_cnt_reset) delay_cnt <= '0;
    else if (!delay_done) delay_cnt <= delay_cnt + DELAY_ONE;
  end

  always_ff @(posedge clk) begin
    if (!reset_n) state <= TX_MAC_IDLE;
    else state <= next;
  end

  always_comb begin
    next = TX_MAC_IDLE;
    if (en)
      unique case(state)
        TX_MAC_IDLE: next = transmit ? TX_MAC_DATA : TX_MAC_IDLE;
        TX_MAC_DATA: next = (!done) ? TX_MAC_DATA : (last_col ? TX_MAC_NEW_LINE_0 : TX_MAC_SPACE);
        TX_MAC_SPACE: next = done ? TX_MAC_DELAY : TX_MAC_SPACE;
        TX_MAC_NEW_LINE_0: next = done ? TX_MAC_NEW_LINE_1 : TX_MAC_NEW_LINE_0;
        TX_MAC_NEW_LINE_1: next = (!done) ? TX_MAC_NEW_LINE_1 : (last_row ? TX_MAC_IDLE :
TX_MAC_DELAY);
        TX_MAC_DELAY: next = delay_done ? TX_MAC_DATA : TX_MAC_DELAY;
        default: next = state;
      endcase
  end

  logic data_load; assign data_load = ((state == TX_MAC_IDLE) & transmit) | ((state == TX_MAC_DELAY) &
delay_done);
  logic space_load; assign space_load = ((state == TX_MAC_DATA) & done & !last_col);
  logic new_line0_load; assign new_line0_load = ((state == TX_MAC_DATA) & done & last_col);
  logic new_line1_load; assign new_line1_load = ((state == TX_MAC_NEW_LINE_0) & done);
  logic any_load; assign any_load = data_load | space_load | new_line0_load | new_line1_load;
  logic [DATA_WIDTH-1:0] next_drive;
  always_comb begin
    unique case(1'b1)
      data_load: next_drive = sw_d.data;
      space_load: next_drive = SPACE_DATA;
      new_line0_load: next_drive = NEW_LINE_0_DATA;
      new_line1_load: next_drive = NEW_LINE_1_DATA;
      default: next_drive = sw_d.data;
    endcase
  end
```

```
    always_ff @(posedge clk) begin
        if (!reset_n) {start, drive_data} <= '0;
        else if (any_load) {start, drive_data} <= {1'b1, next_drive};
        else start <= '0;
    end

    always_ff @(posedge clk) begin
        if (!reset_n) led0 <= '0;
        else if (data_load) led0 <= !led0;
    end

    assign ready = (state == TX_MAC_IDLE);
    assign row_out = row_cnt;
endmodule
```

src/tx_phy.sv

```
`include "lab5_definitions.svh"
import lab5_pkg::*;

module tx_phy
#(
    parameter BAUD_RATE = `BAUD_RATE
) (
    input logic clk, reset_n, start,
    input logic [DATA_WIDTH - 1:0] data,
    output logic tx, done
);

    tx_phy_state_t state, next;

    logic tick;
    baud_gen #(.BAUD_RATE(BAUD_RATE)) baud_gen_i (
        .clk(clk),
        .reset_n(reset_n),
        .en((state == TX_PHY_TRANSMIT)),
        .baud_tick(tick)
    );

    localparam FRAME_BITS_SIZE = $clog2(FRAME_WIDTH);
    localparam [FRAME_BITS_SIZE-1:0] ONE = 1;

    logic [FRAME_BITS_SIZE-1:0] cnt;
    logic [FRAME_WIDTH-1:0] sh_reg;

    assign done = (cnt == FRAME_WIDTH - 1) & tick;
    logic sh_cnt_en, load;
    assign sh_cnt_en = (state == TX_PHY_TRANSMIT) & tick;
    assign load = (state == TX_PHY_IDLE) & start;
    always_ff @(posedge clk) begin
        if (!reset_n) cnt <= '0;
        else if (done | (state == TX_PHY_IDLE)) cnt <= '0;
        else if (sh_cnt_en) cnt <= cnt + ONE;
    end
```

```
    always_ff @(posedge clk) begin
        if (!reset_n) sh_reg <= '1;
        else if (load) sh_reg <= {1'b1, data, 1'b0};
        else if (sh_cnt_en) sh_reg <= {1'b1, sh_reg[FRAME_WIDTH - 1:1]};
    end

    always_ff @(posedge clk) begin
        if (!reset_n) state <= TX_PHY_IDLE;
        else state <= next;
    end

    always_comb begin
        next = state;
        unique case(state)
            TX_PHY_IDLE: next = start ? TX_PHY_TRANSMIT : TX_PHY_IDLE;
            TX_PHY_TRANSMIT: next = done ? TX_PHY_IDLE : TX_PHY_TRANSMIT;
            default: next = state;
        endcase
    end

    assign tx = sh_reg[0];


endmodule
```

src/baud_gen.sv

```
`include "lab5_definitions.svh"
module baud_gen #(
    parameter BAUD_RATE = `BAUD_RATE
) (
    input logic clk, reset_n, en,
    output logic baud_tick
);
    import lab5_pkg::CLK_FREQ;
    localparam COUNT = CLK_FREQ / BAUD_RATE;
    localparam COUNT_BITS = $clog2(COUNT);
    localparam [COUNT_BITS-1:0] ONE = 1;

    logic [COUNT_BITS - 1 : 0] cnt;
    assign baud_tick = (cnt == COUNT - ONE);
    always_ff @(posedge clk) begin
        if (!reset_n) cnt <= '0;
        else if (baud_tick | !en) cnt <= '0;
        else cnt <= cnt + ONE;
    end

endmodule
```

src/rx_phy.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module rx_phy #(
    parameter BAUD_RATE = `BAUD_RATE
) (
    input logic clk, reset_n, en,
    input logic rx,
    output logic [DATA_WIDTH-1:0] data,
    output logic done
);

    localparam TICKS_PER_BIT = 16;
    localparam TICK_CNT = CLK_FREQ / (BAUD_RATE * TICKS_PER_BIT);


    rx_phy_state_t state, next;

    // 1. sync + fall edge
    logic rx_prev, rx_sync, rx_meta;
    always_ff @(posedge clk) begin
        if (!reset_n) {rx_prev, rx_sync, rx_meta} <= '0;
        else {rx_prev, rx_sync, rx_meta} <= {rx_sync, rx_meta, rx};
    end
    logic rx_fall; assign rx_fall = rx_prev && !rx_sync;

    // 2. Cunters
    // tick counter
    localparam TICK_CNT_BITS = (TICK_CNT > 1) ? $clog2(TICK_CNT) : 1;
    localparam [TICK_CNT_BITS-1:0] TICK_ONE = 1;
    localparam [TICK_CNT_BITS-1:0] LAST_CNT = TICK_CNT - 1;
    logic [TICK_CNT_BITS-1:0] tick_cnt;
    logic tick; assign tick = tick_cnt == LAST_CNT;

    always_ff @(posedge clk) begin
        if (!reset_n) tick_cnt <= '0;
        else if (((state == RX_PHY_IDLE) & rx_fall) | tick) tick_cnt <= '0;
        else tick_cnt <= tick_cnt + TICK_ONE;
    end

    // tick16 counter
    localparam TICK16_CNT_BITS = $clog2(TICKS_PER_BIT);
    localparam [TICK16_CNT_BITS-1:0] TICK16_ONE = 1;
    logic [TICK16_CNT_BITS-1:0] tick16_cnt;
    always_ff @(posedge clk) begin
        if (!reset_n) tick16_cnt <= '0;
        else if ((state == RX_PHY_IDLE) & rx_fall) tick16_cnt <= '0;
        else if (tick) tick16_cnt <= tick16_cnt + TICK16_ONE;
    end

    //signals from tick16 counter
    localparam SAMPLE_NUM = 3;
    localparam [TICK16_CNT_BITS-1:0] SAMPLE_TICK = TICKS_PER_BIT / 2 - 1;
    localparam [TICK16_CNT_BITS-1:0] START_CHECK_TICK = SAMPLE_TICK + SAMPLE_NUM;
```

```
localparam [TICK16_CNT_BITS-1:0] LAST_TICK = TICKS_PER_BIT - 1;
logic tick7; assign tick7 = (tick16_cnt == SAMPLE_TICK) & tick;
logic tick10; assign tick10 = (tick16_cnt == START_CHECK_TICK) & tick;
logic tick15; assign tick15 = (tick16_cnt == LAST_TICK) & tick;

logic [SAMPLE_NUM-1:0] start_samples;
always_ff @(posedge clk) begin
   if (!reset_n) start_samples <= '0;
   else if ((state == RX_PHY_START) & tick) start_samples <= {start_samples[SAMPLE_NUM-2:0], rx_sync};
end
logic start_check;
always_ff @(posedge clk) begin
   if (!reset_n) start_check <= '0;
   else if (state == RX_PHY_IDLE) start_check <= '0;
   else if ((!(|start_samples)) & tick10) start_check <= '1;
end

// bit counter
localparam DATA_BITS = $clog2(DATA_WIDTH);
localparam [DATA_BITS-1:0] BIT_ONE = 1;
localparam [DATA_BITS-1:0] LAST_BIT = DATA_WIDTH-1;
logic [DATA_BITS-1:0] bit_cnt;
always_ff @(posedge clk) begin
   if (!reset_n) bit_cnt <= '0;
   else if ((state == RX_PHY_START) & start_check) bit_cnt <= '0;
   else if ((state == RX_PHY_DATA) & tick15) bit_cnt <= bit_cnt + BIT_ONE;
end
logic bit7; assign bit7 = bit_cnt == LAST_BIT;

// 3. fsm
always_ff @(posedge clk) begin
   if (!reset_n) state <= RX_PHY_IDLE;
   else state <= next;
end

always_comb begin
   next = RX_PHY_IDLE;
   if (en) begin
      unique case(state)
         RX_PHY_IDLE: next = (rx_fall) ? RX_PHY_START : RX_PHY_IDLE;
         RX_PHY_START: next = (!tick15) ? RX_PHY_START : (start_check ? RX_PHY_DATA : RX_PHY_IDLE);
         RX_PHY_DATA: next = (bit7 & tick15) ? RX_PHY_STOP : RX_PHY_DATA;
         RX_PHY_STOP: next = (tick7) ? RX_PHY_IDLE : RX_PHY_STOP;
         default: next = state;
      endcase
   end
end

// data_reg
always_ff @(posedge clk) begin
   if (!reset_n) data <= '0;
   else if ((state == RX_PHY_DATA) & tick7) data <= {rx_sync, data[DATA_WIDTH-1:1]};
end

// done reg
always_ff @(posedge clk) begin
```

```
    if (!reset_n) done <= '0;
    else if ((state == RX_PHY_STOP) & tick7) done <= '1;
    else done <= '0;
  end




endmodule
```

src/rx_mac.sv

```
import lab5_pkg::*;

module rx_mac (
  input logic clk, reset_n,
  input logic done,
  input logic [DATA_WIDTH-1:0] data_in,
  output packet_cfg_t pkt_data,
  output logic pkt_valid
);

  rx_mac_state_t state, next;

  localparam [DATA_WIDTH-1:0] ASCII_ZERO = 8'h30;
  localparam [DATA_WIDTH-1:0] ASCII_NINE = 8'h39;
  localparam [DATA_WIDTH-1:0] ASCII_A_BIG = 8'h41;
  localparam [DATA_WIDTH-1:0] ASCII_Z_BIG = 8'h5A;
  localparam [DATA_WIDTH-1:0] ASCII_A_SMALL = 8'h61;
  localparam [DATA_WIDTH-1:0] ASCII_Z_SMALL = 8'h7A;

  logic is_big_char; assign is_big_char = (data_in >= ASCII_A_BIG) & (data_in <= ASCII_Z_BIG);
  logic is_small_char; assign is_small_char = (data_in >= ASCII_A_SMALL) & (data_in <= ASCII_Z_SMALL);

  logic is_comma; assign is_comma = (data_in == COMMA_VAL);
  logic is_start; assign is_start = (data_in == MSG_START_VAL);
  logic is_end; assign is_end = (data_in == MSG_CLOSE_VAL);
  logic is_value; assign is_value = (data_in >= ASCII_ZERO) & (data_in <= ASCII_NINE);
  logic is_opcode; assign is_opcode = is_big_char | is_small_char;

  localparam VALUE_WIDTH = $clog2(NUM_OF_VALUES);
  logic [VALUE_WIDTH-1:0] value_cnt;
  logic value_done; assign value_done = (value_cnt == NUM_OF_VALUES - 1);
  always_ff @(posedge clk) begin
    if (!reset_n) value_cnt <= '0;
    else if ((state == RX_MAC_OPCODE) & done & is_opcode) value_cnt <= '0;
    else if ((state == (RX_MAC_VALUE)) & done & is_value) value_cnt <= value_cnt + 1;
  end

  localparam GROUP_WIDTH = $clog2(MAX_GROUPS);
  logic [GROUP_WIDTH-1:0] group_cnt;
  logic max_group; assign max_group = (group_cnt == MAX_GROUPS - 1);
  always_ff @(posedge clk) begin
    if (!reset_n) group_cnt <= '0;
```

```
    else if (done & is_start) group_cnt <= '0;
    else if ((state == RX_MAC_DONE_NEXT) & done & is_comma & !max_group) group_cnt <= group_cnt + 1;
  end

  always_ff @(posedge clk) begin
    if (!reset_n) state <= RX_MAC_IDLE;
    else state <= next;
  end

  always_comb begin
    next = state;
    unique case(state)
      RX_MAC_IDLE: next = (done & is_start) ? RX_MAC_OPCODE : RX_MAC_IDLE;
      RX_MAC_OPCODE: begin
        if (done) begin
          if (is_start) next = RX_MAC_OPCODE;
          else if (is_opcode) next = RX_MAC_VALUE;
          else next = RX_MAC_IDLE;
        end
      end
      RX_MAC_VALUE: begin
        if (done) begin
          if (is_value) next = (value_done) ? RX_MAC_DONE_NEXT : RX_MAC_VALUE;
          else next = (is_start) ? RX_MAC_OPCODE : RX_MAC_IDLE;
        end
      end
      RX_MAC_DONE_NEXT: begin
        if (done) begin
          if ((!is_comma & !is_start) | (max_group & is_comma)) next = RX_MAC_IDLE;
          else if (is_start | (is_comma & !max_group)) next = RX_MAC_OPCODE;
        end
      end
      default: next = RX_MAC_IDLE;
    endcase
  end


  // MAX_GROUPS is 3, NUM_OF_VALUES = 3; 2 start and end, opcodes as max_group, values the amount of
max_group, and commans is max_group - 1
  // 2 + 3 opcode + 3*3values + 2 commas
  // => PACKET_SIZE = 2 + MAX_GROUPS + MAX_GROUPS*NUM_OF_VALUES + (MAX_GROUPS-1) = 2 + 3 + 9 +
2 = 16
  localparam CALC_PACKET_SIZE = 2 + MAX_GROUPS*(2 + NUM_OF_VALUES) - 1;
  always_ff @(posedge clk) begin
    if (!reset_n) pkt_data <= '0;
    else if (done & is_start) pkt_data <= {{(CALC_PACKET_SIZE-1)*DATA_WIDTH{1'b0}}, MSG_START_VAL};
    else if (done & (((state == RX_MAC_OPCODE) & is_opcode) | ((state == RX_MAC_VALUE) & is_value) |
((state == RX_MAC_DONE_NEXT) & ((is_comma & !max_group) | is_end)))) pkt_data <=
{pkt_data[(CALC_PACKET_SIZE-1)*DATA_WIDTH-1:0], data_in};
  end
```

```
    always_ff @(posedge clk) begin
        if (!reset_n) pkt_valid <= '0;
        else pkt_valid <= (state == RX_MAC_DONE_NEXT) & done & is_end;
    end

endmodule
```

src/parser.sv

```
import lab5_pkg::*;

module parser (
    input logic clk, reset_n,
    input packet_cfg_t pkt_data,
    input logic pkt_valid,
    output rx_parsed_t rx_parsed_data,
    output logic msg_valid // FOR TB!
);

    localparam MAX_VALUE = 1000; // values from 0-999 so 1000 number options
    localparam ASCII_TO_BIN_WIDTH = $clog2(MAX_VALUE);
    localparam [ASCII_TO_BIN_WIDTH-1:0] HUNDRED = 100, TEN = 10;
    localparam [ASCII_TO_BIN_WIDTH-1:0] MAX_PIXEL_VAL = 255;

    logic valid_r; assign valid_r = (pkt_data.r == R_VAL) | (pkt_data.r == R_SMALL_VAL);
    logic valid_c; assign valid_c = (pkt_data.c == C_VAL) | (pkt_data.c == C_SMALL_VAL);
    logic valid_v; assign valid_v = (pkt_data.v == V_VAL) | (pkt_data.v == V_SMALL_VAL);

    // No {, } check because it is already checked in rx_mac, and if small message then c and v and r wont be
in place so msg not valid
    logic valid_opcode; assign valid_opcode = valid_r & valid_c & valid_v;
    logic valid_comma; assign valid_comma = (pkt_data.comma0 == COMMA_VAL) & (pkt_data.comma1 ==
COMMA_VAL);

    // ASCII nums = 0x30 - 0x39 so can take only bottom nibble
    logic [ASCII_TO_BIN_WIDTH-1:0] row, col, val;
    assign row = pkt_data.row_index[2][3:0] * HUNDRED + pkt_data.row_index[1][3:0] * TEN +
pkt_data.row_index[0][3:0];
    assign col = pkt_data.col_index[2][3:0] * HUNDRED + pkt_data.col_index[1][3:0] * TEN +
pkt_data.col_index[0][3:0];
    assign val = pkt_data.value[2][3:0] * HUNDRED + pkt_data.value[1][3:0] * TEN + pkt_data.value[0][3:0];

    logic bad_index; assign bad_index = (row > 255) | (col > 255) | (val > MAX_PIXEL_VAL);
    logic msg_valid_comb; assign msg_valid_comb = valid_comma & valid_opcode & !bad_index;

    always_ff @(posedge clk) begin
        if (!reset_n) msg_valid <= '0;
        else msg_valid <= pkt_valid & msg_valid_comb;
    end
```

```
    always_ff @(posedge clk) begin
        if (!reset_n) rx_parsed_data <= '0;
        else if (pkt_valid & msg_valid_comb) rx_parsed_data <= {row[DATA_WIDTH-1:0], col[DATA_WIDTH-1:0],
val[DATA_WIDTH-1:0]};
    end

endmodule
```

src/display_ctrl.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"
module display_ctrl #(
    parameter DISPLAY_RR = `DISPLAY_RR
) (
    input logic clk, reset_n,
    input rx_parsed_t rx_data,
    input sw_cfg_t tx_data,
    input logic [ROW_SIZE_WIDTH-1:0] row_out,
    input logic rx_tx_mode,
    output display_cfg_t display_out
);

    // 1. tick gen for 500hz
    logic tick;
    baud_gen #(.BAUD_RATE(DISPLAY_RR)) display_tick_i (
        .clk(clk),
        .reset_n(reset_n),
        .en(1'b1),
        .baud_tick(tick)
    );

    // 2. rotate an 0 and 500hz rate
    logic [AN_WIDTH-1:0] an;
    always_ff @(posedge clk) begin
        if (!reset_n) an <= AN_RESET_VAL;
        else if (tick) an <= {an[AN_WIDTH-2:0], an[AN_WIDTH-1]};
    end

    // 3. feed an to rx_display and tx_display
    seven_segment_data_t rx_display_data, tx_display_data;
    rx_display rx_display_i (
        .rx_data(rx_data),
        .an(an),
        .rx_display_data(rx_display_data)
    );

    tx_display tx_display_i (
        .an(an),
        .tx_data(tx_data),
        .row_out(row_out),
        .tx_display_data(tx_display_data)
    );
```

```
    display_ctrl_data_t display_data;

    seven_segment_data_t data_mode;
    assign data_mode = rx_tx_mode ? rx_display_data : tx_display_data;


    seven_segment seven_segment_i (
        .ss_in(data_mode),
        .display_ctrl_data(display_data)
    );

    assign display_out = {an, display_data};

endmodule
```

src/tx_display.sv

```
import lab5_pkg::*;

module tx_display (
    input logic [AN_WIDTH-1:0] an,
    input sw_cfg_t tx_data,
    input logic [ROW_SIZE_WIDTH-1:0] row_out,
    output seven_segment_data_t tx_display_data
);
    localparam DP_DIG = 3;
    //size decode:
    logic [DATA_WIDTH:0] size_val;
    always_comb begin
        unique case(tx_data.size)
            SIZE_1: size_val = SIZE_1_VAL;
            SIZE_32: size_val = SIZE_32_VAL;
            SIZE_128: size_val = SIZE_128_VAL;
            SIZE_256: size_val = SIZE_256_VAL;
            default: size_val = SIZE_1_VAL;
        endcase
    end

    logic [1:0][DIG_IN_WIDTH-1:0] size_d;
    assign size_d = size_val[DATA_WIDTH] ? '1 : size_val[DATA_WIDTH-1:0];

    logic [1:0][DIG_IN_WIDTH-1:0] delay_d;
    always_comb begin
        unique case(tx_data.delay)
            DELAY_0MS: delay_d = DELAY_0MS_DISP;
            DELAY_50MS: delay_d = DELAY_50MS_DISP;
            DELAY_100MS: delay_d = DELAY_100MS_DISP;
            DELAY_200MS: delay_d = DELAY_200MS_DISP;
            default: delay_d = DELAY_0MS_DISP;
        endcase
    end

    logic [1:0][DIG_IN_WIDTH-1:0] row_d; assign row_d = row_out[ROW_SIZE_WIDTH-1] ? '1 :
row_out[DATA_WIDTH-1:0];
```

```
  always_comb begin
    tx_display_data.dp = an[DP_DIG];
    tx_display_data.dash = '0;
    unique case(1'b0)
      an[0]: tx_display_data.data_val = tx_data.data[0];
      an[1]: tx_display_data.data_val = tx_data.data[1];
      an[2]: tx_display_data.data_val = delay_d[0];
      an[3]: tx_display_data.data_val = delay_d[1];
      an[4]: tx_display_data.data_val = size_d[0];
      an[5]: tx_display_data.data_val = size_d[1];
      an[6]: tx_display_data.data_val = row_d[0];
      an[7]: tx_display_data.data_val = row_d[1];
      default: tx_display_data.data_val = '0;
    endcase
  end

endmodule
```

src/rx_display.sv

```
import lab5_pkg::*;

module rx_display (
  input rx_parsed_t rx_data,
  input logic [AN_WIDTH-1:0] an,
  output seven_segment_data_t rx_display_data
);
  always_comb begin
    rx_display_data = {1'b0, {DIG_IN_WIDTH{1'b0}}, 1'b1};
    if (!an[2] | !an[3]) rx_display_data.dash = '1;
    unique case(1'b0)
      an[0]: rx_display_data.data_val = rx_data.pixel_value[0];
      an[1]: rx_display_data.data_val = rx_data.pixel_value[1];
      an[2]: rx_display_data.data_val = '0;
      an[3]: rx_display_data.data_val = '0;
      an[4]: rx_display_data.data_val = rx_data.col_index[0];
      an[5]: rx_display_data.data_val = rx_data.col_index[1];
      an[6]: rx_display_data.data_val = rx_data.row_index[0];
      an[7]: rx_display_data.data_val = rx_data.row_index[1];
      default: rx_display_data.data_val = '0;
    endcase
  end
endmodule
```

src/seven_segment.sv

```
import lab5_pkg::*;

module seven_segment (
  input seven_segment_data_t ss_in,
  output display_ctrl_data_t display_ctrl_data
);
```

```
  always_comb begin
    display_ctrl_data.dp = ss_in.dp;
    if (ss_in.dash) display_ctrl_data.ss_dig = DASH_VAL;
    else unique case (ss_in.data_val)
      4'h0: display_ctrl_data.ss_dig = 7'b0000001;
      4'h1: display_ctrl_data.ss_dig = 7'b1001111;
      4'h2: display_ctrl_data.ss_dig = 7'b0010010;
      4'h3: display_ctrl_data.ss_dig = 7'b0000110;
      4'h4: display_ctrl_data.ss_dig = 7'b1001100;
      4'h5: display_ctrl_data.ss_dig = 7'b0100100;
      4'h6: display_ctrl_data.ss_dig = 7'b0100000;
      4'h7: display_ctrl_data.ss_dig = 7'b0001111;
      4'h8: display_ctrl_data.ss_dig = 7'b0000000;
      4'h9: display_ctrl_data.ss_dig = 7'b0000100;
      4'hA: display_ctrl_data.ss_dig = 7'b0001000;
      4'hB: display_ctrl_data.ss_dig = 7'b1100000;
      4'hC: display_ctrl_data.ss_dig = 7'b0110001;
      4'hD: display_ctrl_data.ss_dig = 7'b1000010;
      4'hE: display_ctrl_data.ss_dig = 7'b0110000;
      4'hF: display_ctrl_data.ss_dig = 7'b0111000;
      default: display_ctrl_data.ss_dig = 7'b1111111;
    endcase
  end
endmodule
```

NexysA7-100T_Master.xdc (האילוצים הפעילים בלבד)

```
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35
Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clk}];
set_false_path -from [get_ports {cpu_reset btnc sw[*] rx}]
set_false_path -to [get_ports {tx led0 led15 an[*] seg[*] dp}]
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { sw[0] }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { sw[1] }]; #IO_L3N_T0_DQS_EMCCLK_14
Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { sw[2] }]; #IO_L6N_T0_D08_VREF_14
Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { sw[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { sw[4] }]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { sw[5] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { sw[6] }]; #IO_L17N_T2_A13_D29_14
Sch=sw[6]
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { sw[7] }]; #IO_L5N_T0_D07_14 Sch=sw[7]
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { sw[8] }]; #IO_L24N_T3_34 Sch=sw[8]
set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { sw[9] }]; #IO_25_34 Sch=sw[9]
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { sw[10] }]; #IO_L20P_T3_A08_D24_14
Sch=sw[13]
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { sw[11] }]; #IO_L19N_T3_A09_D25_VREF_14
Sch=sw[14]
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { sw[12] }]; #IO_L21P_T3_DQS_14
Sch=sw[15]
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { led0 }]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN V11   IOSTANDARD LVCMOS33 } [get_ports { led15 }]; #IO_L21N_T3_DQS_A06_D22_14
Sch=led[15]
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { seg[6] }]; #IO_L24N_T3_A00_D16_14
Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { seg[5] }]; #IO_25_14 Sch=cb
```

```
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { seg[4] }]; #IO_25_15 Sch=cc
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { seg[3] }]; #IO_L17P_T2_A26_15 Sch=cd
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { seg[2] }]; #IO_L13P_T2_MRCC_14 Sch=ce
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { seg[1] }]; #IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { seg[0] }]; #IO_L4P_T0_D04_14 Sch=cg
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { dp }]; #IO_L19N_T3_A21_VREF_15 Sch=dp
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { an[0] }]; #IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { an[1] }]; #IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { an[2] }]; #IO_L24P_T3_A01_D17_14
Sch=an[2]
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { an[3] }]; #IO_L19P_T3_A22_15 Sch=an[3]
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { an[4] }]; #IO_L8N_T1_D12_14 Sch=an[4]
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { an[5] }]; #IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { an[6] }]; #IO_L23P_T3_35 Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { an[7] }]; #IO_L23N_T3_A02_D18_14
Sch=an[7]
set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { cpu_reset }]; #IO_L3P_T0_DQS_AD1P_15
Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { btnc }]; #IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN C4    IOSTANDARD LVCMOS33 } [get_ports { rx }]; #IO_L7P_T1_AD6P_35 Sch=uart_txd_in
set_property -dict { PACKAGE_PIN D4    IOSTANDARD LVCMOS33 } [get_ports { tx }]; #IO_L11N_T1_SRCC_35
Sch=uart_rxd_out
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

נספח ב': קוד הטסטבנצ'ים

הטסטבנצ'ים לפי הסדר של פרק 4:

sim/tb_debounce.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module tb_debounce;
  timeunit 1ns;
  timeprecision 1ps;
  localparam DEB_WAIT = (`DEB_COUNT + 2 + 1) * 10; // 2 for sync + 1 for state reg
  bit clk = '0;
  always #5 clk = !clk;

  logic reset_n = '0;
  logic io_in = '0;
  logic io_d;

  debounce #(.DEB_COUNT(`DEB_COUNT)) debounce_i (
    .clk(clk),
    .reset_n(reset_n),
    .io_in(io_in),
    .io_d(io_d)
  );

  initial $monitor("Time: %0t | io_in=%0b | io_d=%0b", $time, io_in, io_d);

  task reset_dut();
    $display("Time: %0t | TASK: Reset DUT", $time);
    reset_n = '0; #(CLK_PERIOD);
    assert(!io_d);
    reset_n = '1;
  endtask

  bit toggle_value;

  task toggle_io(input bit t_val);
    $display("Time: %0t | TASK: Toggle io to: %0b", $time, t_val);
    io_in = t_val;
    toggle_value = t_val;
  endtask

  task wait_after_toggle(input int wait_time);
    $display("Time: %0t | TASK: Wait for: %0d ns", $time, wait_time);
    #(wait_time);
  endtask


  initial begin
    reset_dut();

    // 1. Regular
    $display("Time: %0t | Toggle io to 1", $time);
    toggle_io(1); wait_after_toggle(DEB_WAIT); assert(io_d);
```

```
        $display("Time: %0t | Toggle io to 0", $time);
        toggle_io(0); wait_after_toggle(DEB_WAIT); assert(!io_d);

        #(CLK_PERIOD);

        //2. Debounce Teset
        $display("Time: %0t | Debounce teset", $time);
        toggle_io(1); wait_after_toggle(DEB_WAIT / 3); assert(!io_d);
        toggle_io(0); wait_after_toggle(DEB_WAIT / 6); assert(!io_d);
        toggle_io(1); wait_after_toggle(DEB_WAIT / 10); assert(!io_d);
        toggle_io(0); wait_after_toggle(DEB_WAIT / 5); assert(!io_d);
        toggle_io(1); wait_after_toggle(DEB_WAIT); assert(io_d);

        #(CLK_PERIOD);

        //3. Reset mid
        toggle_io(0); wait_after_toggle(DEB_WAIT / 2); assert(io_d);
        reset_dut(); assert(!io_d);
        toggle_io(1); wait_after_toggle(DEB_WAIT); assert(io_d);

        #(CLK_PERIOD);

        $finish;
    end

    property p_io_d_reset_val;
        @(posedge clk)
        (!reset_n) |=> (!io_d);
    endproperty

    assert property(p_io_d_reset_val);

endmodule
```

sim/tb_debounce_top.sv

```
import lab5_pkg::*;

module tb_debounce_top;
    timeunit 1ns;
    timeprecision 1ps;

    bit clk = '0;
    always #5 clk = !clk;

    logic cpu_reset = '0;
    logic cpu_reset_d;
    board_in_t io_in = '0;
    board_in_t io_d;

    debounce_top debounce_top_i (
        .clk(clk),
        .cpu_reset(cpu_reset),
        .io_in(io_in),
```

```
        .io_d(io_d),
        .cpu_reset_d(cpu_reset_d)
    );

    localparam DEB_COUNT = `DEB_COUNT;
    localparam SYNC_WAIT = 2 * CLK_PERIOD;
    localparam COUNT_WAIT = DEB_COUNT * CLK_PERIOD;
    localparam LATCH_WAIT = 1 * CLK_PERIOD;

    localparam TOTAL_WAIT_FOR_CHANGE = SYNC_WAIT + COUNT_WAIT + LATCH_WAIT;
    localparam RESET_WAIT_FOR_CHANGE = DEB_COUNT + 1;

    initial $monitor("Time: %5d | cpu_reset=%b | cpu_reset_d=%b | io_in=%b | io_d=%b", $time, cpu_reset,
cpu_reset_d, io_in, io_d);

    logic [13:0] io_cfg = {{13{1'b0}}, {1'b1}};
    integer i;
    initial begin
        cpu_reset = '1; #(TOTAL_WAIT_FOR_CHANGE);

        // show that every board_t input has a debounce
        for (i = 0; i < 14; i++) begin
            io_in = io_cfg << i; #(TOTAL_WAIT_FOR_CHANGE);
        end

        // show reset debounce
        $display("Time: %5d | RESET Test", $time);
        cpu_reset = '0; #(TOTAL_WAIT_FOR_CHANGE);
        cpu_reset = '1; #(TOTAL_WAIT_FOR_CHANGE * 2);

        // io_cfg is now 10000..
        // show debounce efect only working when pressed (changed for more than COUNT_WAIT)
        $display("Time: %5d | DEBOUNCE Test", $time);
        io_in = '0; #(COUNT_WAIT/4);
        io_in = '1; #(COUNT_WAIT/6);
        io_in = '0; #(COUNT_WAIT/2);
        io_in = '1; #(COUNT_WAIT - 1);
        io_in = '0; #(COUNT_WAIT);
        #(SYNC_WAIT + CLK_PERIOD + LATCH_WAIT);
        $finish;
    end

    property reset_cnt_reset;
    @(posedge clk)
    ($past(debounce_top_i.reset_edge)) |-> debounce_top_i.reset_cnt == '0;
    endproperty
    assert property(reset_cnt_reset);

    property reset_cnt_no_wrap;
        @(posedge clk)
```

```
      ($past(debounce_top_i.reset_stable) & !$past(debounce_top_i.reset_edge)) |->
debounce_top_i.reset_cnt == DEB_COUNT-1;
   endproperty
   assert property(reset_cnt_no_wrap);



endmodule
```

sim/tb_btn_hold.sv

```
`include "lab5_definitions.svh"
module tb_btn_hold;
   timeunit 1ns;
   timeprecision 1ps;
   import lab5_pkg::*;

   localparam BTN_HOLD_WAIT = (`BTN_HOLD_COUNT) * 10;

   bit clk = '0;
   always #5 clk =!clk;

   logic reset_n;
   logic btn = '0;
   logic held;

   btn_hold #(.BTN_HOLD_COUNT(`BTN_HOLD_COUNT)) btn_hold_i (
      .clk(clk),
      .reset_n(reset_n),
      .btn(btn),
      .held(held)
   );

   initial $monitor("Time: %0t | btn=%0b | held=%0b", $time, btn, held);

   task reset_dut();
      $display("Time: %0t | TASK: Reset DUT", $time);
      reset_n = '0; #(CLK_PERIOD); assert(!held);
      reset_n = '1;
   endtask

   task press_button(input int press_time, input bit release_press);
      $display("Time: %0t | TASK: Press button", $time);
      btn = '1; #(press_time);
      if (press_time == (BTN_HOLD_WAIT)) assert(held);
      else assert(!held); #(CLK_PERIOD);
      if (release_press) begin
         $display("Time: %0t | TASK: Release button press", $time);
         btn = '0; #(CLK_PERIOD);
      end
   endtask

   initial begin
      reset_dut();
```

```
    $display("Time: %0t | Regular press", $time);
    press_button(BTN_HOLD_WAIT, '1);

    $display("Time: %0t | Short press", $time);
    press_button(BTN_HOLD_WAIT/2, '1);

    $display("Time: %0t | Long press", $time);
    press_button(BTN_HOLD_WAIT * 2, '1);

    $display("Time: %0t | Reset mid press", $time);
    press_button(BTN_HOLD_WAIT/2, '0);
    reset_dut();
    press_button(BTN_HOLD_WAIT, '1);


    $finish;
end


// Functionality:
// 1. btn pressed for BTN_HOLD_COUNT cycles => held = '1;
// 2. if btn is pressed for period > BTN_HOLD_WAIT then only one held pulse should be recorded
// 3. held is a one pulse signal

// 1.

int count_press;
always @(posedge clk) begin
    if (!reset_n | !btn) count_press <= '0;
    else if (btn) count_press <= count_press + 1;
end

bit valid_press; assign valid_press = (count_press == `BTN_HOLD_COUNT);

property p_held_after_count;
    @(posedge clk)
    disable iff (!reset_n)
    (valid_press) |-> held;
endproperty

// 2.
int count_held;
always_ff @(posedge clk) begin
    if (!reset_n | !btn) count_held <= '0;
    else if (held) count_held <= count_held + 1;
end

bit one_held; assign one_held = (count_held == 1);

property p_held_once_per_btn;
    @(posedge clk)
    disable iff (!reset_n)
    (count_press > `BTN_HOLD_COUNT) |-> (one_held);
endproperty
```

```
    // 3.
    property p_held_one_cycle;
        @(posedge clk)
        disable iff(!reset_n)
        ($past(held)) |-> (!held);
    endproperty

    assert property(p_held_after_count);
    assert property(p_held_once_per_btn);
    assert property(p_held_one_cycle);


endmodule
```

sim/tb_latch.sv

```
import lab5_pkg::*;

module tb_latch;
    timeunit 1ns;
    timeprecision 1ps;
    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic ready = '0;
    sw_cfg_t sw_d = '0;
    logic held = '0;
    logic rx_tx_mode = '0;
    sw_cfg_t sw_cfg;
    logic transmit;

    latch latch_i(
        .clk(clk),
        .reset_n(reset_n),
        .ready(ready),
        .sw_d(sw_d),
        .held(held),
        .rx_tx_mode(rx_tx_mode),
        .sw_cfg(sw_cfg),
        .transmit(transmit)
    );

    initial $monitor("Time: %0t | sw_cfg=%b | transmit=%b",
                $time, sw_cfg, transmit);

    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD);
        assert((!transmit) & (sw_cfg === '0));
        reset_n = '1;
    endtask

    task set_sw_d(input logic [11:0] sw_val);
        $display("Time: %0t | TASK: Set sw_d to: %b", $time, sw_val);
```

```
    sw_d = sw_val;
  endtask

  task set_mode(input logic mode);
    $display("Time: %0t | TASK: Set rx_tx_mode to: %s (%b)", $time, (mode) ? "RX" : "TX", mode);
    rx_tx_mode = mode;
  endtask

  task set_ready(input logic r);
    $display("Time: %0t | TASK: Set ready to: %b", $time, r);
    ready = r;
  endtask

  task set_held(input logic h);
    $display("Time: %0t | TASK: Set held to: %b", $time, h);
    held = h;
  endtask

  integer i;

  initial begin
    $display("Time: %0t | Reset DUT", $time);
    reset_dut();

    set_sw_d('1);

    for (i=0; i<2**3; i++) begin
      set_mode(i[2]);
      set_held(i[1]);
      set_ready(i[0]);
      #(CLK_PERIOD);
    end

    #(CLK_PERIOD);
    $finish;
  end


  property p_latch_en;
    @(posedge clk)
    disable iff(!reset_n)
    ($past(ready) & $past(held) & $past(!rx_tx_mode)) |-> (sw_cfg === sw_d);
  endproperty

  property p_reset_values;
    @(posedge clk)
    (!reset_n) |=> ({transmit, sw_cfg} === '0);
  endproperty

  assert property(p_latch_en);
  assert property(p_reset_values);


endmodule
```

sim/tb_tx_mac.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module tb_tx_mac;

    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic en = '0;
    sw_cfg_t sw_d = '0;
    logic transmit = '0;
    logic done;
    logic [DATA_WIDTH-1:0] drive_data;
    logic start, led0;
    logic [ROW_SIZE_WIDTH-1:0] row_out;
    logic ready;

    tx_mac dut (
        .clk(clk),
        .reset_n(reset_n),
        .en(en),
        .sw_d(sw_d),
        .transmit(transmit),
        .done(done),
        .drive_data(drive_data),
        .start(start),
        .led0(led0),
        .row_out(row_out),
        .ready(ready)
    );

    bit exp_led0;
    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD);
        assert ((drive_data === '0) & (led0 === '0) & (start === '0) & (row_out === '0) & (ready === '1));
        reset_n = '1;
    endtask

    // acting like latch
    task send_data(input sw_cfg_t sw);
        $display("Time: %0t | TASK: Drive inputs: data=%b, size=%b, delay=%b",
                    $time, sw.data, sw.size, sw.delay);
        {transmit, sw_d} = { 1'b1 , sw}; #(CLK_PERIOD);
        transmit = '0;
    endtask

    task en_dut(input logic e);
        $display("Time %0t | TASK: %s DUT", $time, e ? "Enable" : "Disable");
        en = e;
    endtask

    // the way i am going to check every byte is going to be like this:
```

```
  /*
    A counter the counts from zero to (ALL_BYTES_SIZE - 1).
    ALL_BYTES_SIZE is calculated like this:
      in every row their are n data bytes, (n-1) space bytes and 2 new line bytes
        in total n + (n-1) + 2 = 2n + 1
      in total their are n line too so:
        ALL_BYTES_SIZE = n * (2n + 1) | n = SIZE from sw_d

    now each placment of the byte has a specific data value:
        0     1     2     3     4
      0 data  space data  /r    /n
      1 data  space data  /r    /n

    from this example:
      1. last 2 col in a row always /r /n
      2. if not 2 last cols and col is even -> data
      3. if not 2 last cols and col is odd -> space

    To calculate the position of row and col of each bytes based on a running counter,
    I'll use:
      - col = i % (2n + 1)
      - row = i / (2n + 1)
      * i is the running counter index

    since their are in total from 0 : (2n + 1 -1), (2n:0), col to count
    for i in range ALL_BYTES_SIZE
      row = i / 2n+1
      col = i % 2n+1
      if (col == 2n-1) exp_drive_data = /r
      if (col == 2n) exp_drive_data = /n
      if (col % 2 == 0)
        exp_drive_data = data
        exp_led0 = !exp_led0
      else exp_drive_data = space
      assert(row === row_out) // row calculation is just row_out

      at the start i will have a exp_led0, that will reset on !resetn
      and have a:
      assert((even col) & led0 === exp_led0)

    counter counts on each posedge of start.
    And the counter needs to restart when i tell it (not on transmit because transmit doesnt mean new
transmittion)
  */

  function int get_size(input size_cfg_t size);
    case(size)
      SIZE_1: get_size = SIZE_1_VAL;
      SIZE_32: get_size = SIZE_32_VAL;
      SIZE_128: get_size = SIZE_128_VAL;
      SIZE_256: get_size = SIZE_256_VAL;
      default: get_size = SIZE_1_VAL;
    endcase
  endfunction

  localparam MAX_SIZE = SIZE_256_VAL * (2 * SIZE_256_VAL + 1);
```

```
    logic [$clog2(MAX_SIZE)-1:0] byte_cnt;
    bit reset_byte_cnt;
    bit last_byte; assign last_byte = (byte_cnt == get_size(sw_d.size));
    logic [DATA_WIDTH - 1:0] exp_byte; assign exp_byte = get_expected_byte(byte_cnt, sw_d.data,
get_size(sw_d.size));
    bit exp_led0_passive; assign exp_led0_passive = (is_data_byte(byte_cnt, get_size(sw_d.size))) ? !exp_led0 :
exp_led0;
    always_ff @(posedge clk) begin
        if (!reset_n) begin
            byte_cnt <= '0;
            exp_led0 <= '0;
        end
        else if (reset_byte_cnt | !en) byte_cnt <= '0;
        else if (start) begin
            //$display("Time: %0t| drive_data=%b | exp_drive_data=%b", $time, drive_data, exp_byte);
            assert(drive_data === exp_byte);
            assert(row_out === get_row(byte_cnt, get_size(sw_d.size)));
            byte_cnt <= byte_cnt + 1;
            if (is_data_byte(byte_cnt, get_size(sw_d.size))) exp_led0 <= exp_led0_passive;
            //$display("Time: %0t | led0=%b | exp_led0=%b", $time, led0, exp_led0);
            assert(led0 === exp_led0_passive);
        end
    end

    function logic [ROW_SIZE_WIDTH-1:0] get_row(input logic [$clog2(MAX_SIZE)-1:0] i, input int n);
        int col_size;
        col_size = 2*n + 1;
        return i / col_size;
    endfunction


    function bit is_data_byte(input logic [$clog2(MAX_SIZE)-1:0] i, input int n);
        int col;
        bit not_new_line;
        col = i % (2*n + 1);
        not_new_line = (col < 2*n-1);
        return (not_new_line) & (col%2 ==0);
    endfunction

    function logic [DATA_WIDTH-1:0] get_expected_byte (input logic [$clog2(MAX_SIZE)-1:0] idx, input logic
[DATA_WIDTH-1:0] data, input int s);
        int col, col_size;
        col_size = (2 * s + 1);
        col = idx % col_size;
        if (col == col_size-2) return NEW_LINE_0_DATA;
        if (col == col_size-1) return NEW_LINE_1_DATA;
        if (col % 2 == 0) return data;
        else return SPACE_DATA;
    endfunction

    task wait_to_reset_byte_cnt();
        int size;
        size = get_size(sw_d.size);
        while(!ready) #(CLK_PERIOD);
        assert(byte_cnt == size * (2 * size + 1));
        reset_byte_cnt = 1; #(CLK_PERIOD);
```

```
    reset_byte_cnt = '0;
  endtask


  // need to make phy interaction for mac
  // phy:
  // when start goes up for one cycle,
  // after one (CLKS_PER_BITS * 10) mac gets a dont input,
  // to shorten the tb i will make (CLKS_PER_BIT * 10) 1 clk cycle.
  // added option to change the delay time
  logic start_state;
  logic [$clog2((CLK_FREQ/`BAUD_RATE)*10)-1:0] cnt;

  always_ff @(posedge clk) begin
    if (!reset_n) cnt <= '0;
    else if (!start_state) cnt <= '0;
    else cnt <= cnt + 1;
  end
  localparam DELAY_CYCLES = 1;
  assign done = (cnt == (DELAY_CYCLES - 1)) & start_state;

  always_ff @(posedge clk) begin
    if (!reset_n) begin
      start_state <= '0;
    end else if (start) start_state <= '1;
    else if (done) begin
      start_state <= '0;
    end
  end

  initial begin
    reset_dut();

    en_dut('1);
    send_data({SIZE_32, DELAY_0MS, 8'h41});
    wait_to_reset_byte_cnt();

    send_data({SIZE_128, DELAY_50MS, 8'hAA});
    wait_to_reset_byte_cnt();

    send_data({SIZE_256, DELAY_100MS, 8'hAA});
    wait_to_reset_byte_cnt();

    send_data({SIZE_1, DELAY_200MS, 8'hBB});
    wait_to_reset_byte_cnt();

    send_data({SIZE_128, DELAY_50MS, 8'hFF});
    #(((SIZE_128_VAL * (2 * SIZE_128_VAL + 1)) / 2) * CLK_PERIOD);
    en_dut(0); #(CLK_PERIOD)
    en_dut(1);

    send_data({SIZE_128, DELAY_50MS, 8'hA5});
    #(((SIZE_128_VAL * (2 * SIZE_128_VAL + 1)) / 2) * CLK_PERIOD);
    send_data({SIZE_128, DELAY_50MS, 8'hA5}); // sw_d has to stay the same during a run (latch
functionality)
    wait_to_reset_byte_cnt();
```

```
        send_data({SIZE_1, DELAY_200MS, 8'h0D});
        wait_to_reset_byte_cnt();


        $finish;
    end

endmodule
```

sim/tb_tx_phy.sv

```
import lab5_pkg::*;

module tb_tx_phy;
    timeunit 1ns;
    timeprecision 1ps;
    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic start = '0;
    logic [DATA_WIDTH-1:0] data = '0;
    logic tx, done;

    tx_phy t_phy_i(
        .clk(clk),
        .reset_n(reset_n),
        .start(start),
        .data(data),
        .tx(tx),
        .done(done)
    );

    // How many clks per bit?
    // CLK_FREQ / BAUD_RATE
    localparam CLKS_PER_BIT = CLK_FREQ / `BAUD_RATE;
    localparam BIT_WAIT = CLKS_PER_BIT * CLK_PERIOD;

    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD); assert((tx === '1) & (done === '0));
        reset_n = '1;
    endtask

    logic [FRAME_WIDTH-1:0] frame;
    task send_data(input logic [DATA_WIDTH-1:0] data_2_drive);
        $display("Time: %0t | TASK: Send data: %b", $time, data_2_drive);
        data = data_2_drive; start = '1; #(CLK_PERIOD);
        start = '0;
        frame = {1'b1, data_2_drive, 1'b0};
        for (int i=0; i < FRAME_WIDTH; i++) begin
```

```
        if (i == 0) #(BIT_WAIT - 1*CLK_PERIOD); // To take in to consideration the first #(CLK_PERIOD) after
start
        else #(BIT_WAIT);
        $display("Time: %0t | TASK: DUT_tx=%b, TB_tx=%b", $time, tx, frame[i]);
        assert(frame[i] === tx);
      end
      assert(done);
      #(CLK_PERIOD);
    endtask

    task check_idle(input int clk_cycles);
      $display("Time: %0t | TASK: Check it tx=1 while idle", $time);
      for (int i = 0; i < clk_cycles; i++) begin
        #(CLK_PERIOD);
        assert((tx === '1) & (done === '0));
      end
    endtask

    initial $monitor("Time: %0t | tx=%b | done=%b",
                     $time, tx, done);

    initial begin
      $display("Time: %0t | Reset DUT", $time);
      reset_dut();

      // Drive 0xA5;
      $display("Time: %0t | Driving: %b", $time, 8'hA5);
      send_data(8'hA5);

      // Drive 0x5A;
      $display("Time: %0t | Driving: %b", $time, 8'h5A);
      send_data(8'h5A);

      //Drive 0x00
      $display("Time: %0t | Driving: %b", $time, 8'h00);
      send_data(8'h00);

      //Drive 0xFF
      $display("Time: %0t | Driving: %b", $time, 8'hFF);
      send_data(8'hff);

      check_idle(100);

      $finish;
    end


    property p_load;
      @(posedge clk)
      disable iff(!reset_n)
      ($past(start)) |-> (t_phy_i.sh_reg == {1'b1, data, 1'b0});
    endproperty

    property p_tx_one_done;
      @(posedge clk)
      disable iff(!reset_n)
```

```
    ($past(done)) |-> (tx);
  endproperty

  assert property(p_load);
  assert property(p_tx_one_done);

endmodule
```

sim/tb_baud_gen.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module tb_baud_gen;
  timeunit 1ns;
  timeprecision 1ps;
  localparam BAUD_COUNT = (CLK_FREQ/`BAUD_RATE);
  localparam TICK_WAIT = (BAUD_COUNT - 1) * CLK_PERIOD;
  localparam HALF_WAIT = (BAUD_COUNT / 2) * CLK_PERIOD;

  bit clk = '0;
  always #5 clk =!clk;

  bit reset_n, en, baud_tick;

  baud_gen #(.BAUD_RATE(`BAUD_RATE)) baud_gen_i (
    .clk(clk),
    .reset_n(reset_n),
    .en(en),
    .baud_tick(baud_tick)
  );

  task reset_dut(input bit en_reset);
    $display("Time: %0t | TASK: Reset DUT", $time);
    reset_n = '0;
    if (en_reset) en = '0;
    #(CLK_PERIOD);
    reset_n = '1;
  endtask

  task en_baud_gen(input int wait_time, input bit lower_en);
    $display("Time: %0t | TASK: Raise enable for %0d ns", $time, wait_time);
    en = '1; #(wait_time);
    if (lower_en) begin
      $display("Time: %0t | TASK: Lower enable", $time);
      en = '0;
    end
  endtask

  initial $monitor("Time: %0t | en=%0b | baud_tick=%0b", $time, en, baud_tick);

  initial begin
    reset_dut('1);
    assert(!baud_tick);
```

```
    en_baud_gen(TICK_WAIT, '1);
    assert(baud_tick); #(CLK_PERIOD);

    en_baud_gen(HALF_WAIT, '1);
    assert(!baud_tick); #(CLK_PERIOD);

    $display("Time: %0t | Reset mid en", $time);
    en_baud_gen(HALF_WAIT, '0);
    assert(!baud_tick);
    reset_dut('0);
    assert(!baud_tick);
    en_baud_gen(TICK_WAIT, '1);
    assert(baud_tick);
    #(CLK_PERIOD); $finish;

  end


  int count;
  always_ff @(posedge clk) begin
    if (!reset_n | !en | (count == BAUD_COUNT-1)) count <= '0;
    else if (en) count <= count + 1;
  end

  bit valid_en; assign valid_en = (count == BAUD_COUNT-1);

  property p_tick_after_count;
    @(posedge clk)
    disable iff (!reset_n)
    (valid_en) |-> (baud_tick);
  endproperty
  assert property(p_tick_after_count);

  property p_baud_tick_one_cycle;
    @(posedge clk)
    disable iff(!reset_n)
    ($past(baud_tick)) |-> (!baud_tick);
  endproperty
  assert property(p_baud_tick_one_cycle);

  property p_reset_press;
    @(posedge clk)
    (!reset_n) |=> (baud_tick == '0);
  endproperty
  assert property(p_reset_press);



endmodule
```

sim/tb_rx_phy.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"
module tb_rx_phy;

    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic en = '0;
    logic rx = '1;
    logic [DATA_WIDTH-1:0] data;
    logic done;

    rx_phy dut (
        .clk(clk),
        .reset_n(reset_n),
        .en(en),
        .rx(rx),
        .data(data),
        .done(done)
    );

    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD);
        assert(data === '0);
        assert(done === '0);
        reset_n = '1;
    endtask

    function logic [FRAME_WIDTH-1:0] get_frame (input logic [DATA_WIDTH-1:0] data);
        return {1'b1, data, 1'b0};
    endfunction

    localparam BIT_WAIT_CYCLES = CLK_FREQ / `BAUD_RATE;
    localparam BIT_WAIT = BIT_WAIT_CYCLES * CLK_PERIOD;
    localparam MID_BIT_WAIT = BIT_WAIT / 2;
    localparam SYNC_WAIT = 3 * CLK_PERIOD; // From waves


    bit sent_frame;
    task send_frame(input [DATA_WIDTH-1:0] d_in);
        logic [FRAME_WIDTH-1:0] frame;
        sent_frame = '1;
        frame = get_frame(d_in);
        $display("Time: %0t | TASK: Sending frame: %b", $time, frame);
        for (int i = 0; i < FRAME_WIDTH; i++) begin
            $display("Time: %0t | TASK: Sending bit[%0d] | rx = %b", $time, i, frame[i]);
            rx = frame[i];
            if (i == FRAME_WIDTH-1) begin
                #(MID_BIT_WAIT + SYNC_WAIT);
                assert(done);
                assert(data === d_in);
                #(BIT_WAIT - (MID_BIT_WAIT + SYNC_WAIT));
```

```
      end else #(BIT_WAIT);
    end
    sent_frame = '0;
    #(CLK_PERIOD);
endtask

task send_frame_not_en(input [DATA_WIDTH-1:0] d_in, input logic [DATA_WIDTH-1:0] prev_d_in);
    logic [FRAME_WIDTH-1:0] frame;
    sent_frame = '1;
    frame = get_frame(d_in);
    $display("Time: %0t | TASK: Sending frame: %b", $time, frame);
    for (int i = 0; i < FRAME_WIDTH; i++) begin
      $display("Time: %0t | TASK: Sending bit[%0d] | rx = %b", $time, i, frame[i]);
      rx = frame[i];
      if (i == FRAME_WIDTH-1) begin
        #(MID_BIT_WAIT + SYNC_WAIT);
        assert(!done);
        assert(data === {d_in[3:0], prev_d_in[7:4]});
        #(BIT_WAIT - (MID_BIT_WAIT + SYNC_WAIT));
      end else if (i == 5) begin
        en_dut('0);
        en_dut('1);
        #(BIT_WAIT - 2*CLK_PERIOD);
      end else #(BIT_WAIT);
    end
    sent_frame = '0;
    #(CLK_PERIOD);
endtask

task en_dut(input bit e);
    $display("Time: %0t | TASK: %s DUT", $time, e ? "Enable" : "Disable");
    en = e; #(CLK_PERIOD);
endtask

initial begin
    reset_dut();
    en_dut('1);
    send_frame(8'hA5);
    send_frame(8'h5A);
    send_frame_not_en(8'h84, 8'h5A);
    send_frame(8'hFF);
    send_frame(8'h00);

    $finish;
end

property p_done_one_cyle;
    @(posedge clk)
    disable iff(!reset_n)
    ($past(done)) |-> (!done);
endproperty
assert property(p_done_one_cyle);

// done once per frame
int cnt;
```

```
    always_ff @(posedge clk) begin
        if (!reset_n) cnt <= '0;
        else if (sent_frame & done) cnt <= cnt + 1;
        else if (!sent_frame) cnt <= '0;
        assert(cnt <= 1);
    end

    property p_done_low_not_in_frame;
        @(posedge clk)
        disable iff(!reset_n)
        (!sent_frame) |-> (done == '0);
    endproperty
    assert property(p_done_low_not_in_frame);




endmodule
```

sim/tb_rx_mac.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module tb_rx_mac;

    bit clk = '0;
    always #5 clk = !clk;

    logic reset_n = '0;
    logic done = '0;
    logic [DATA_WIDTH-1:0] data_in = '0;
    packet_cfg_t pkt_data;
    logic pkt_valid;

    rx_mac dut (
        .clk(clk),
        .reset_n(reset_n),
        .done(done),
        .data_in(data_in),
        .pkt_data(pkt_data),
        .pkt_valid(pkt_valid)
    );

    task reset_dut();
        $display("Time: %0t | TASK: Reset DUT", $time);
        reset_n = '0; #(CLK_PERIOD);
        assert(!pkt_valid);
        assert(pkt_data === '0);
        reset_n = '1;
    endtask

    // full flow : {RXXX,CXXX,VXXX}
    task drive_full_input(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
```

```
    for (int i = 0; i < 16; i++) begin
      case(i)
        0: data_in = MSG_START_VAL;
        1: data_in = R_VAL;
        2: data_in = row_val[2];
        3: data_in = row_val[1];
        4: data_in = row_val[0];
        5, 10: data_in = COMMA_VAL;
        6: data_in = C_VAL;
        7: data_in = col_val[2];
        8: data_in = col_val[1];
        9: data_in = col_val[0];
        11: data_in = V_VAL;
        12: data_in = val[2];
        13: data_in = val[1];
        14: data_in = val[0];
        15: data_in = MSG_CLOSE_VAL;
        default: data_in = MSG_START_VAL;
      endcase
      done = '1; #(CLK_PERIOD);
      if (i == 15) begin
        assert(pkt_valid === '1);
        //$display("Time: %0t | %h", $time, build_valid_pkt(row_val, col_val, val));
        assert(pkt_data === build_exp_pkt(row_val, col_val, val));
      end
      done = '0; #(CLK_PERIOD);
    end
  endtask

  task drive_full_input_bad(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
    for (int i = 0; i < 17; i++) begin
      case(i)
        0: data_in = MSG_START_VAL;
        1: data_in = R_VAL;
        2: data_in = row_val[2];
        3: data_in = row_val[1];
        4: data_in = row_val[0];
        5, 10: data_in = COMMA_VAL;
        6: data_in = C_VAL;
        7: data_in = col_val[2];
        8: data_in = col_val[1];
        9: data_in = col_val[0];
        11: data_in = V_VAL;
        12: data_in = val[2];
        13: data_in = val[1];
        14: data_in = val[0];
        15: data_in = MSG_START_VAL;
        16: data_in = MSG_CLOSE_VAL;
        default: data_in = MSG_START_VAL;
      endcase
      done = '1; #(CLK_PERIOD);
      if (i == 15) begin
        assert(pkt_valid === '0);
        //$display("Time: %0t | %h", $time, build_exp_pkt(row_val, col_val, val));
        assert(pkt_data !== build_exp_pkt(row_val, col_val, val));
      end
```

```
        done = '0; #(CLK_PERIOD);
      end
    endtask

    task drive_restart_input(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
      for (int i = 0; i < 20; i++) begin
        case(i)
          0, 4: data_in = MSG_START_VAL;
          1, 5: data_in = R_VAL;
          2, 6: data_in = row_val[2];
          3, 7: data_in = row_val[1];
          8: data_in = row_val[0];
          9, 14: data_in = COMMA_VAL;
          10: data_in = C_VAL;
          11: data_in = col_val[2];
          12: data_in = col_val[1];
          13: data_in = col_val[0];
          15: data_in = V_VAL;
          16: data_in = val[2];
          17: data_in = val[1];
          18: data_in = val[0];
          19: data_in = MSG_CLOSE_VAL;
          default: data_in = MSG_START_VAL;
        endcase
        done = '1; #(CLK_PERIOD);
        if (i == 19) begin
          assert(pkt_valid === '1);
          assert(pkt_data === build_exp_pkt(row_val, col_val, val));
        end
        done = '0; #(CLK_PERIOD);
      end
    endtask

    task drive_one_group_input(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val);
      for (int i = 0; i < 6; i++) begin
        case(i)
          0: data_in = MSG_START_VAL;
          1: data_in = R_VAL;
          2: data_in = row_val[2];
          3: data_in = row_val[1];
          4: data_in = row_val[0];
          5: data_in = MSG_CLOSE_VAL;
          default: data_in = MSG_START_VAL;
        endcase
        done = '1; #(CLK_PERIOD);
        if (i == 5) begin
          assert(pkt_valid === '1);
          assert(pkt_data === {{10*DATA_WIDTH{1'b0}}, MSG_START_VAL, R_VAL, row_val,
MSG_CLOSE_VAL});
        end
        done = '0; #(CLK_PERIOD);
      end
    endtask

    task drive_four_groups_input(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
      for (int i = 0; i < 21; i++) begin
```

```
        case(i)
          0: data_in = MSG_START_VAL;
          1, 16: data_in = R_VAL;
          2, 17: data_in = row_val[2];
          3, 18: data_in = row_val[1];
          4, 19: data_in = row_val[0];
          5, 10, 15: data_in = COMMA_VAL;
          6: data_in = C_VAL;
          7: data_in = col_val[2];
          8: data_in = col_val[1];
          9: data_in = col_val[0];
          11: data_in = V_VAL;
          12: data_in = val[2];
          13: data_in = val[1];
          14: data_in = val[0];
          20: data_in = MSG_CLOSE_VAL;
          default: data_in = MSG_START_VAL;
        endcase
        done = '1; #(CLK_PERIOD);
        if (i >= 15) begin
          assert(pkt_valid === '0);
        end
        done = '0; #(CLK_PERIOD);
      end
    endtask

    function packet_cfg_t build_exp_pkt(input logic [DIGITS-1:0][DATA_WIDTH-1:0] row_val, col_val, val);
      packet_cfg_t p;
      p.msg_start = MSG_START_VAL;
      p.r         = R_VAL;
      p.row_index = row_val;
      p.comma0    = COMMA_VAL;
      p.c         = C_VAL;
      p.col_index = col_val;
      p.comma1    = COMMA_VAL;
      p.v         = V_VAL;
      p.value     = val;
      p.msg_close = MSG_CLOSE_VAL;
      return p;
    endfunction

    initial begin
      reset_dut();

      drive_full_input(32'h313233, 32'h343536, 32'h373839);

      #(CLK_PERIOD * 5);

      drive_full_input_bad(32'h090807, 32'h060504, 32'h030201);
      #(CLK_PERIOD * 5);

      drive_restart_input(32'h323535, 32'h303030, 32'h313238);
      #(CLK_PERIOD * 5);

      drive_one_group_input(32'h313233);
      #(CLK_PERIOD * 5);
```

```
    drive_four_groups_input(32'h313233, 32'h343536, 32'h373839);
    #(CLK_PERIOD * 5);

    $finish;
  end

endmodule
```

sim/tb_parser.sv

```
import lab5_pkg::*;

module tb_parser;

  bit clk = '0;
  always #5 clk = !clk;

  logic reset_n = '0;
  packet_cfg_t pkt_data = '0;
  logic pkt_valid = '0;
  rx_parsed_t rx_parsed_data;
  logic msg_valid;

  parser dut (
    .clk(clk),
    .reset_n(reset_n),
    .pkt_data(pkt_data),
    .pkt_valid(pkt_valid),
    .rx_parsed_data(rx_parsed_data),
    .msg_valid(msg_valid)
  );

  task reset_dut();
    $display("Time: %0t | TASK: Reset DUT", $time);
    reset_n = '0; #(CLK_PERIOD);
    assert(rx_parsed_data === '0);
    assert(msg_valid === '0);
    reset_n = '1;
  endtask


  function logic [DIGITS-1:0][DATA_WIDTH-1:0] hex_to_ascii(input logic [SCREENS_PER_SEGMENT-
1:0][DIG_IN_WIDTH-1:0] x);
    logic [DIGITS-1:0][DATA_WIDTH-1:0] temp;
    temp[2] = "0" + x / 100;
    temp[1] = "0" + (x/10) % 10;
    temp[0] = "0" + x % 10;
    return temp;
  endfunction

  function packet_cfg_t convert_parsed_to_packet(input logic [2:0][DIGITS-1:0][DATA_WIDTH-1:0] x);
    return {"{", pkt_data.r , x[2], ",", pkt_data.c , x[1], ",", pkt_data.v , x[0], "}"};
  endfunction
```

```
function packet_cfg_t full_conversion_to_packet(input rx_parsed_t x);
   logic [DIGITS-1:0][DATA_WIDTH-1:0] row,col,val;
   row = hex_to_ascii(x.row_index);
   col = hex_to_ascii(x.col_index);
   val = hex_to_ascii(x.pixel_value);
   return convert_parsed_to_packet({row, col, val});
endfunction

task drive_pkt_data(input packet_cfg_t pkt);
   $display("Time: %0t | TASK: Drive pkt_data with: %h", $time, pkt);
   pkt_data = pkt;
endtask;

task drive_pkt_valid();
   $display("Time: %0t | TASK: Drive pkt_valid to 1 cycle", $time);
   pkt_valid = '1; #(CLK_PERIOD);
   pkt_valid = '0;
endtask


task drive_flow (input packet_cfg_t pkt);
   drive_pkt_data(pkt);
   drive_pkt_valid();
   assert(msg_valid === '1);
   assert(pkt_data === full_conversion_to_packet(rx_parsed_data));
   #(CLK_PERIOD);
endtask

task drive_false_flow(input packet_cfg_t pkt);
   rx_parsed_t temp;
   temp = rx_parsed_data;
   drive_pkt_data(pkt);
   drive_pkt_valid();
   assert(msg_valid === '0);
   assert(rx_parsed_data === temp);
   #(CLK_PERIOD);
endtask


initial begin
   reset_dut();

   // First check regular
   drive_flow("{R123,C045,V200}");
   drive_flow("{R255,C255,V255}");

   drive_false_flow("{A111,B222,X444}");

   drive_false_flow("{R256,C000,V000}");
   drive_false_flow("{R000,C256,V000}");
   drive_false_flow("{R000,C000,V256}");

   drive_flow("{r005,c050,v055}");
   drive_flow("{R123,c089,V111}");
```

```
    drive_false_flow("{R123;C045,V200}");

    $finish;


  end



endmodule
```

sim/tb_rx_display.sv

```
import lab5_pkg::*;

module tb_rx_display;

  rx_parsed_t rx_data;
  logic [AN_WIDTH-1:0] an;
  seven_segment_data_t rx_display_data;

  rx_display dut (
    .rx_data(rx_data),
    .an(an),
    .rx_display_data(rx_display_data)
  );

  function [AN_WIDTH-1:0] gen_an(input int i);
    return ~(1 << i);
  endfunction

  function bit is_dash(input int i);
    return (i == 2) || (i == 3);
  endfunction

  function [DIG_IN_WIDTH-1:0] exp_val(input rx_parsed_t p, input int i);
    unique case(i)
      0: return p.pixel_value[0];
      1: return p.pixel_value[1];
      2, 3: return '0;
      4: return p.col_index[0];
      5: return p.col_index[1];
      6: return p.row_index[0];
      7: return p.row_index[1];
      default: return 'x;
    endcase
  endfunction

  task drive(input rx_parsed_t p);
    rx_data = p;
    for (int i = 0; i < AN_WIDTH; i++) begin
      an = gen_an(i);
      #(CLK_PERIOD);
```

```
            assert(rx_display_data.dash === is_dash(i));
            assert(rx_display_data.dp === '1);
            if (!is_dash(i))
                assert(rx_display_data.data_val === exp_val(p, i));
        end
    endtask

    initial begin
        drive(24'hABCABC);
        drive(24'h123456);
        $finish;
    end



endmodule
```

sim/tb_tx_display.sv

```
import lab5_pkg::*;

module tb_tx_display;

    logic [AN_WIDTH-1:0] an = '0;
    sw_cfg_t tx_data = '0;
    logic [ROW_SIZE_WIDTH-1:0] row_out = '0;
    seven_segment_data_t tx_display_data;

    tx_display dut (
        .an(an),
        .tx_data(tx_data),
        .row_out(row_out),
        .tx_display_data(tx_display_data)
    );



    function logic [AN_WIDTH-1:0] get_an(input int i);
        return ~(1 << i);
    endfunction

    function sw_cfg_t load_tx_data(input size_cfg_t size, input delay_cfg_t delay, input logic
[1:0][DIG_IN_WIDTH-1:0] data);
        return {size, delay, data};
    endfunction


    task test_drive(input size_cfg_t size, input delay_cfg_t delay, input logic [1:0][DIG_IN_WIDTH-1:0] data,
input logic [ROW_SIZE_WIDTH-1:0] row);
        int i;
        for (i = 0; i < AN_WIDTH; i++) begin
            an = get_an(i);
            row_out = row;
            tx_data = load_tx_data(size, delay, data);
```

```
        #(CLK_PERIOD);
      end

   endtask

   int s, d;
   initial begin
      for (s = 0; s < 4; s++) begin
         for (d = 0; d < 4; d++) begin
            test_drive(size_cfg_t'(s), delay_cfg_t'(d), 8'hA5, (s+d));
         end
      end
      $finish;
   end


endmodule
```

sim/tb_seven_segment.sv

```
import lab5_pkg::*;

module tb_seven_segment;
   timeunit 1ns;
   timeprecision 1ps;
   seven_segment_data_t ss_in;
   display_ctrl_data_t display_ctrl_data;

   seven_segment ss_i(
      .ss_in(ss_in),
      .display_ctrl_data(display_ctrl_data)
   );

   initial $monitor("Time: %5d | dash=%b | data_val=%b | dp_in=%b | ss_dig=%b | dp=%b",
            $time, ss_in.dash, ss_in.data_val, ss_in.dp, display_ctrl_data.ss_dig, display_ctrl_data.dp);

   localparam SS_IN_WIDTH = 6;
   integer i;

   initial begin
      ss_in = '0; #(CLK_PERIOD);
      for (i = 0; i<(2**SS_IN_WIDTH)/2 +2; i++) begin
         ss_in = i[SS_IN_WIDTH-1:0]; #(CLK_PERIOD);
      end

      $finish;
   end

endmodule
```

sim/tb_display_ctrl.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module tb_display_ctrl;

  bit clk = '0;
  always #5 clk = !clk;

  logic reset_n = '0;
  rx_parsed_t rx_data = '0;
  sw_cfg_t tx_data = '0;
  logic [ROW_SIZE_WIDTH-1:0] row_out = '0;
  logic rx_tx_mode = '0;
  display_cfg_t display_out;

  display_ctrl dut (
    .clk(clk),
    .reset_n(reset_n),
    .rx_data(rx_data),
    .tx_data(tx_data),
    .row_out(row_out),
    .rx_tx_mode(rx_tx_mode),
    .display_out(display_out)
  );

  localparam CYCLES_PER_AN = CLK_FREQ / `DISPLAY_RR;
  localparam AN_WAIT = CLK_PERIOD * CYCLES_PER_AN;

  task reset_dut();
    $display("Time: %0t | Task: Reset DUT", $time);
    reset_n = '0; #(CLK_PERIOD);
    assert(display_out.an === AN_RESET_VAL);
    assert(display_out.seg === SEG_RESET_VAL);
    assert(display_out.dp === DP_RESET_VAL);
    reset_n = '1;
  endtask

  function [AN_WIDTH-1:0] exp_an(input int i);
    return ~(1 << i);
  endfunction

  task drive(input rx_parsed_t r, input sw_cfg_t t, input logic[ROW_SIZE_WIDTH-1:0] row, input logic mode);
    $display("Time: %0t | TASK: Driving inputs: rx_data=%h, tx_data=%h, row_out=%h, rx_tx_mode=%b",
             $time, r, t, row, mode);
    rx_data = r;
    tx_data = t;
    row_out = row;
    rx_tx_mode = mode;
    for (int i = 0; i < AN_WIDTH; i++) begin
      assert(display_out.an === exp_an(i));
      #(AN_WAIT);
    end
  endtask
```

```
    initial begin
        reset_dut();

        drive(24'h123456, {SIZE_128, DELAY_50MS, 8'hA5}, 9'd77, '0);

        #(CLK_PERIOD * 5);
        $finish;
    end

endmodule
```

sim/tb_chip_top.sv

```
import lab5_pkg::*;
`include "lab5_definitions.svh"

module tb_chip_top;

    bit clk = '0;
    always #5 clk = !clk;

    logic cpu_reset = '0;
    logic [SW_WIDTH-1:0] sw = '0;
    logic btnc = '0;
    logic rx = '1;
    logic tx;
    logic [AN_WIDTH-1:0] an;
    logic [SS_DIG_WIDTH-1:0] seg;
    logic dp;
    logic led0, led15;

    chip_top dut (
        .clk(clk),
        .cpu_reset(cpu_reset),
        .sw(sw),
        .btnc(btnc),
        .rx(rx),
        .tx(tx),
        .an(an),
        .seg(seg),
        .dp(dp),
        .led0(led0),
        .led15(led15)
    );

    localparam DEB_WAIT = (`DEB_COUNT + 5) * CLK_PERIOD;
    localparam HOLD_WAIT = (`DEB_COUNT + `BTN_HOLD_COUNT + 10) * CLK_PERIOD;
    localparam BIT_WAIT = (CLK_FREQ / `BAUD_RATE) * CLK_PERIOD;
    localparam FRAME_WAIT = FRAME_WIDTH * BIT_WAIT;

    task reset_dut();
        $display("Time: %0t | Task: Reset DUT", $time);
        cpu_reset = '0; #(DEB_WAIT);
        cpu_reset = '1; #(DEB_WAIT);
    endtask
```

```
    task set_sw(input logic mode, input size_cfg_t size, input delay_cfg_t delay, input logic [DATA_WIDTH-1:0]
data);
        $display("Time: %0t | TASK: Set switches: mode=%b, size=%s, delay=%s, data=%h",
                 $time, mode, size.name(), delay.name(), data);
        sw = {mode, size, delay, data};
        #(DEB_WAIT);
    endtask

    task press_btnc();
        $display("Time: %0t | TASK: Press BTNC", $time);
        btnc = '1; #(HOLD_WAIT);
        btnc = '0; #(DEB_WAIT);
    endtask

    task send_byte(input logic [DATA_WIDTH-1:0] data);
        rx = '0; #(BIT_WAIT);
        for (int i = 0; i < DATA_WIDTH; i++) begin
            rx = data[i]; #(BIT_WAIT);
        end
        rx = '1; #(BIT_WAIT);
    endtask

    task send_msg(input logic [PACKET_SIZE*DATA_WIDTH-1:0] msg);
        $display("Time: %0t | TASK: Send message: %s", $time, msg);
        for (int i = PACKET_SIZE-1; i >= 0; i--) begin
            send_byte(msg[i*DATA_WIDTH +: DATA_WIDTH]);
        end
        #(BIT_WAIT);
    endtask

    task wait_frames(input int n);
        $display("Time: %0t | TASK: Wait %0d frames", $time, n);
        #(FRAME_WAIT * n);
    endtask

    initial begin
        reset_dut();

        set_sw('0, SIZE_1, DELAY_0MS, 8'h41);
        press_btnc();
        wait_frames(5);

        set_sw('0, SIZE_32, DELAY_50MS, 8'h5A);
        press_btnc();
        wait_frames(2200);

        set_sw('1, SIZE_1, DELAY_0MS, 8'h00);
        send_msg("{R001,C002,V128}");
        send_msg("{r010,c020,v255}");
        send_msg("{R300,C005,V009}");
```

```
        press_btnc();

        #(CLK_PERIOD * 5);
        $finish;
    end

endmodule
```